Quantum Machine Learning

Assessment of QML: encodings, kernels, variational models and the QCNN

ANSWER BOOK

This book accompanies the separate *Question Paper* (80 questions, 3 hours, 240 marks). It contains

1. the **answer key**, section by section, with the topic tested by every question (page 2),
2. a **topic-wise map** grouping the questions by theme and a **quick-marking grid** (page 4), and
3. the **Answer Bank**: every question restated with the correct option marked ✓ and a complete solution with derivations (from page 5).

Marking scheme (every question)

Each question has **four** options (A), (B), (C) and (D). **ONLY ONE** of these four options is correct.

Full marks:	+3 if ONLY the correct option is chosen;
Zero marks:	0 if none of the options is chosen (question unanswered);
Negative marks:	−1 in all other cases (wrong option, or more than one option chosen).

How to evaluate

For each section count N_c = number of correct answers and N_w = number of questions answered wrongly (or with more than one option marked). Section marks $= 3N_c - N_w$; unanswered questions score 0. The total is out of 240. Every numerical answer was re-computed independently (NumPy, exact rational arithmetic where possible, an independent NumPy state-vector simulator, and CUDA-Q for the circuits and for the QCNN kernel of `QML.ipynb`) by `verify_answers.py` in the source folder.

Conventions (identical to the Question Paper)

- Qubits are numbered $0, 1, \ldots$; a basis state $|q_0 q_1 \ldots\rangle$ lists **qubit 0 first** (as in CUDA-Q's bitstrings), and two-qubit matrices use the basis order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$.
- Gates: $H = \frac{1}{\sqrt{2}}\left(\begin{smallmatrix}1 & 1\\ 1 & -1\end{smallmatrix}\right)$; X, Y, Z are the Pauli matrices; $S = \mathrm{diag}(1, i)$; $R_P(\theta) = e^{-i\theta P/2}$ for $P \in \{X, Y, Z\}$, so $R_Y(\theta)|0\rangle = \cos\frac{\theta}{2}|0\rangle + \sin\frac{\theta}{2}|1\rangle$; CNOT (control c, target t) flips t when c is 1; a controlled-U applies U to the target when the control is $|1\rangle$.
- Measurements are in the computational basis; $\langle M\rangle = \langle\psi|M|\psi\rangle$, so $\langle Z\rangle = P(0) - P(1)$; the Bloch vector is $(\langle X\rangle, \langle Y\rangle, \langle Z\rangle)$. Two-term parameter-shift rule: $f'(\theta) = \frac{1}{2}\left[f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})\right]$.
- Machine learning: labels $y \in \{+1, -1\}$ with prediction $\mathrm{sign}\, f$ unless stated ($y \in \{0, 1\}$ for cross-entropy); MSE $\mathcal{L} = \frac{1}{M}\sum_m (f_m - y_m)^2$; $\sigma(z) = 1/(1 + e^{-z})$; precision $= \frac{TP}{TP+FP}$, recall $= \frac{TP}{TP+FN}$, $F_1 = \frac{2PR}{P+R}$; ln is the natural logarithm. Numerical answers are rounded.
- **Notebook** (`QML.ipynb`, defaults): 2×4 images with one line of 2 lit pixels (horizontal = +1, vertical = −1), background noise in $[0, 0.25]$, 20 training / 20 test images, `SEED = 42`; one qubit per pixel; encoding $R_Y(\pi x_i)$. QCNN layer $\ell = 1, 2, \ldots$ has 4 shared parameters $\theta_{\ell,1..4}$ = `theta[4(ℓ-1)]` … `theta[4(ℓ-1)+3]`: convolution block $= \mathrm{CNOT}_{a\to b}\left(R_Y(\theta_{\ell,1})_a \otimes R_Y(\theta_{\ell,2})_b\right)$ on pairs $(0,1), (2,3), \ldots$ then $(1,2), (3,4), \ldots$; pooling block (source a, sink b) $= |1\rangle\langle 1|_a \otimes R_Z(\theta_{\ell,3}) + |0\rangle\langle 0|_a \otimes R_X(\theta_{\ell,4})$, then a is discarded. Readout $\langle Z_{\mathrm{out}}\rangle$ with `cudaq.observe` (exact); MSE loss; COBYLA with 100 loss evaluations.
- **Previously recorded run** = outputs stored in the notebook (CUDA-Q 0.16.0, `qpp-cpu`, no GPU): 12 parameters, 69 gates, depth 21, 1.7 ms per circuit, 3.4 s training, loss 1.1214 → 0.6704, accuracy 90 % (training) / 85 % (test).

Answer key

Answer distribution: (A) 20, (B) 20, (C) 20, (D) 20.

Section A — Classical Machine Learning and Mathematics

Q.	Ans.	Topic tested	Q.	Ans.	Topic tested
1	**C**	One gradient-descent step	11	**B**	Standard error of an accuracy
2	**D**	Binary cross-entropy	12	**B**	Data leakage and test-set tuning
3	**C**	Precision, recall, F1	13	**C**	Gradient of the MSE (matrix form)
4	**C**	PCA from the covariance matrix	14	**A**	Momentum
5	**D**	Bayes' theorem; precision at low prevalence	15	**A**	Derivatives of activations
6	**D**	Chain rule for one neuron	16	**B**	Over-/underfitting, early stopping, imbalance
7	**B**	Softmax and cross-entropy	17	**A**	XOR and linear separability
8	**B**	Parameter count of a dense network	18	**C**	GD step-size limit; effect of L_2 penalty
9	**D**	Gaussian (RBF) kernel	19	**A**	Forward pass of a small network
10	**D**	Kronecker product	20	**A**	Back-propagation; dead ReLU

Section B — Quantum Computing Foundations

Q.	Ans.	Topic tested	Q.	Ans.	Topic tested
21	**B**	Expectation values from amplitudes	31	**A**	Conjugation $R_Y^\dagger Z R_Y$
22	**B**	Pauli algebra	32	**D**	Controlled-R_Z matrix
23	**D**	Period of $f(\theta)$; global phase	33	**A**	State-vector memory
24	**D**	Product vs. entangled states	34	**A**	Two-parameter single-qubit circuit
25	**C**	Bell-state correlations	35	**C**	Gate identities
26	**C**	Local rotation on an entangled qubit	36	**D**	Bloch vectors
27	**B**	$\langle Z \rangle$ from shots; standard error	37	**C**	Same statistics, different states
28	**C**	Tensor product; qubit ordering	38	**D**	CNOT and the product-state criterion
29	**B**	Unitarity	39	**A**	Correlations of R_Y + CNOT state
30	**A**	Global vs. relative phase	40	**B**	Purity and maximal entanglement

Section C — Quantum Machine Learning Fundamentals

Q.	Ans.	Topic tested	Q.	Ans.	Topic tested
41	**A**	Angle encoding of a pixel	51	**B**	QCNN gate count
42	**D**	Amplitude encoding	52	**A**	MSE vs. accuracy
43	**C**	Qubit cost of encodings	53	**A**	Circuit count and run time of the notebook
44	**B**	Fidelity kernel of angle encoding	54	**B**	Data re-uploading frequencies
45	**A**	Size of the kernel computation	55	**C**	Encodings and feature maps
46	**D**	Swap test	56	**C**	What each QML model trains
47	**C**	Two-term parameter-shift rule	57	**B**	Gradient-free training; shift-rule validity
48	**C**	Shared parameters in the QCNN	58	**A**	PSD kernel matrices; shot noise
49	**D**	QCNN depth and parameters	59	**B**	VQC output (Heisenberg picture)
50	**D**	Pooling block (code)	60	**D**	VQC gradient by parameter shift

Section D — Advanced QML and Implementation

Q.	Ans.	Topic tested	Q.	Ans.	Topic tested
61	**B**	Barren plateau: shots needed	71	**A**	Quantum PCA
62	**A**	Barren plateaus: what is known	72	**C**	GAN equilibrium
63	**A**	Two- vs. four-term shift rule	73	**C**	Q-learning update
64	**B**	Depolarising noise and accuracy	74	**B**	Swap-test distances (clustering)
65	**D**	GPU memory and multi-GPU qubits	75	**C**	CUDA-Q targets
66	**D**	Cost of an exact QCNN gradient	76	**D**	Limitations of QML
67	**A**	Expectation values from counts	77	**D**	Quantum-advantage claims
68	**A**	Interpreting MSE 0.67 at 90 % accuracy	78	**B**	Recorded hardware; target selection
69	**B**	15-qubit training time	79	**C**	Recorded run: precision, recall, F1
70	**C**	Kernel concentration	80	**D**	Recorded run vs. classical baselines

SILICO FELLER

Topic-wise map of the questions

Every question appears under at least one theme; several appear under two.

Theme	Question numbers
Linear algebra: vectors, matrices, eigenvalues, tensor products	4, 10, 13, 22, 28, 29, 31
Probability, statistics and sampling	5, 11, 27, 61, 67, 70, 80
Derivatives, gradient descent and optimizers	1, 6, 13, 14, 18, 20, 57, 66
Neural networks, activations and loss functions	2, 7, 8, 15, 17, 19, 20, 52
Model evaluation, data handling and diagnosis	3, 11, 12, 16, 68, 79, 80
Qubits, states, Bloch sphere and measurement	21, 23, 27, 30, 33, 36, 37
Gates, circuits and parameterized circuits	29, 31, 32, 34, 35, 39, 50
Entanglement and correlations	24, 25, 26, 38, 39, 40
Data encoding and feature maps	36, 41, 42, 43, 54, 55
Kernels, quantum kernels and kernel methods	9, 44, 45, 46, 58, 70, 74
Variational models, QNN, VQC and hybrid training	34, 47, 52, 53, 56, 59, 60
Parameter-shift rules and gradient cost	47, 48, 57, 60, 63, 66
The QCNN of `QML.ipynb`: layout, code, counts	48, 49, 50, 51, 53, 66, 68
Barren plateaus, noise and trainability	26, 61, 62, 64
Other QML algorithms: QSVM, qPCA, QGAN, QRL, clustering	56, 70, 71, 72, 73, 74
CUDA-Q, GPU simulation and performance	33, 65, 67, 69, 75, 78
Limitations, benchmarking and quantum-advantage claims	44, 76, 77, 79, 80

Quick-marking grid

1 **C**	2 **D**	3 **C**	4 **C**	5 **D**	6 **D**	7 **B**	8 **B**	9 **D**	10 **D**
11 **B**	12 **B**	13 **C**	14 **A**	15 **A**	16 **B**	17 **A**	18 **C**	19 **A**	20 **A**
21 **B**	22 **B**	23 **D**	24 **D**	25 **C**	26 **C**	27 **B**	28 **C**	29 **B**	30 **A**
31 **A**	32 **D**	33 **A**	34 **A**	35 **C**	36 **D**	37 **C**	38 **D**	39 **A**	40 **B**
41 **A**	42 **D**	43 **C**	44 **B**	45 **A**	46 **D**	47 **C**	48 **C**	49 **D**	50 **D**
51 **B**	52 **A**	53 **A**	54 **B**	55 **C**	56 **C**	57 **B**	58 **A**	59 **B**	60 **D**
61 **B**	62 **A**	63 **A**	64 **B**	65 **D**	66 **D**	67 **A**	68 **A**	69 **B**	70 **C**
71 **A**	72 **C**	73 **C**	74 **B**	75 **C**	76 **D**	77 **D**	78 **B**	79 **C**	80 **D**

SILICO FELLER

Answer Bank — questions with detailed solutions

SECTION A · Classical Machine Learning and Mathematics Q.1–20

Q.1 SINGLE-CORRECT MCQ · One gradient-descent step

The loss $L(w_1, w_2) = (w_1 - 3)^2 + 2(w_2 + 1)^2$ is minimised by gradient descent with learning rate $\eta = 0.1$, starting from $\mathbf{w} = (0, 0)$. After *one* step the parameters and the loss are

(A) $(0.6,\ -0.8)$ and $L = 5.84$
(B) $(-0.6,\ 0.4)$ and $L = 16.88$
(C) ✓ $(0.6,\ -0.4)$ and $L = 6.48$
(D) $(0.6,\ -0.4)$ and $L = 11$

Answer: (C) Detailed solution

Correct answer. **(C)** $(0.6,\ -0.4)$ and $L = 6.48$
Concepts required. Gradient descent moves the parameters a small step *against* the gradient; the gradient of a sum of squares is computed term by term.
Relevant formulae. $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w})$, η = learning rate; $\frac{d}{dw} c(w - a)^2 = 2c(w - a)$.
Step-by-step explanation. The gradient is $\nabla L = (2(w_1 - 3),\ 4(w_2 + 1))$; at $(0, 0)$ it equals $(-6,\ 4)$. Gradient descent moves *against* the gradient: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L = (0, 0) - 0.1\,(-6, 4) = (0.6,\ -0.4)$. New loss: $L = (0.6 - 3)^2 + 2(-0.4 + 1)^2 = 5.76 + 2(0.36) = 5.76 + 0.72 = 6.48$, down from $L(0, 0) = 9 + 2 = 11$.
Why the other options are incorrect.
(A) $(0.6, -0.8)$ uses $8(w_2 + 1)$ instead of $4(w_2 + 1)$ for the derivative of $2(w_2 + 1)^2$.
(B) $(-0.6, 0.4)$ is a step *along* the gradient $(\mathbf{w} + \eta \nabla L)$; the loss rises from 11 to 16.88.
(D) The point is right but 11 is the loss *before* the step, $L(0, 0) = 9 + 2$.
Key takeaway. One GD step: compute ∇L at the current point, subtract $\eta \nabla L$, then re-evaluate the loss.

Q.2 SINGLE-CORRECT MCQ · Binary cross-entropy

A binary classifier outputs the probabilities $p = 0.9,\ 0.6,\ 0.2$ (probability of class 1) for three examples whose true labels are $y = 1,\ 1,\ 0$. The *mean* binary cross-entropy (natural logarithm) is closest to

(A) 0.742 **(B)** 0.122 **(C)** 0.839 **(D)** ✓ 0.280

Answer: (D) Detailed solution

Correct answer. **(D)** 0.280
Concepts required. Binary cross-entropy rewards a high probability for the *correct* class; for label 0 the relevant probability is $1 - p$.
Relevant formulae. $\ell = -[y \ln p + (1 - y) \ln(1 - p)]$, mean $\bar{\ell} = \frac{1}{M} \sum_m \ell_m$; ln = natural log.
Step-by-step explanation. Per example $\ell = -[y \ln p + (1 - y) \ln(1 - p)]$: $\ell_1 = -\ln 0.9 = 0.1054$, $\ell_2 = -\ln 0.6 = 0.5108$, $\ell_3 = -\ln(1 - 0.2) = -\ln 0.8 = 0.2231$ (label 0, so the probability of the *correct* class is $1 - p = 0.8$). Mean: $(0.1054 + 0.5108 + 0.2231)/3 = 0.8393/3 = 0.2798 \approx 0.280$.
Why the other options are incorrect.
(A) 0.742 uses $-\ln 0.2$ for the third example, i.e. treats its label as 1.
(B) 0.122 uses $\log_{10}$ instead of ln.
(C) 0.839 is the *sum* of the three losses, not the mean.
Key takeaway. Cross-entropy = $-\ln$(probability assigned to the true class), averaged over the examples.

Q.3 SINGLE-CORRECT MCQ · Precision, recall, F1

On a test set a classifier gives TP = 40, FP = 10, FN = 20 and TN = 130. Its F1-score is

(A) 0.850 **(B)** 0.800 **(C)** ✓ 0.727 **(D)** 0.733

Answer: (C) Detailed solution

Correct answer. **(C)** 0.727

Concepts required. Precision: how many flagged items are right; recall: how many true items are found; F1 combines them harmonically.

Relevant formulae. $P = \frac{TP}{TP+FP}$, $R = \frac{TP}{TP+FN}$, $F_1 = \frac{2PR}{P+R} = \frac{2TP}{2TP+FP+FN}$, accuracy $= \frac{TP+TN}{\text{all}}$.

Step-by-step explanation. Precision $P = \frac{TP}{TP+FP} = \frac{40}{50} = 0.8$; recall $R = \frac{TP}{TP+FN} = \frac{40}{60} = 0.667$. F1 is the *harmonic* mean: $F_1 = \frac{2PR}{P+R} = \frac{2(0.8)(0.667)}{1.467} = \frac{1.0667}{1.4667} = 0.727$ (equivalently $F_1 = \frac{2TP}{2TP+FP+FN} = \frac{80}{110}$). For comparison the accuracy is $\frac{170}{200} = 0.85$.

Why the other options are incorrect.

(A) 0.850 is the accuracy $(40+130)/200$, inflated by the many true negatives.

(B) 0.800 is the precision alone.

(D) 0.733 is the *arithmetic* mean $(0.8+0.667)/2$; F1 is the harmonic mean, which is never larger.

Key takeaway. $F_1 = 2TP/(2TP+FP+FN)$ ignores true negatives, which is why it is preferred for imbalanced data.

Q.4 SINGLE-CORRECT MCQ · PCA from the covariance matrix

Centred two-dimensional data have the covariance matrix $C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. The first principal component explains which fraction of the total variance, and along which direction does it point?

(A) 50 %, along $(1, 0)$ **(B)** 67 %, along $(1, 1)/\sqrt{2}$

(C) ✓ 75 %, along $(1, 1)/\sqrt{2}$ **(D)** 75 %, along $(1, -1)/\sqrt{2}$

Answer: (C) Detailed solution

Correct answer. **(C)** 75 %, along $(1, 1)/\sqrt{2}$

Concepts required. PCA: the principal directions are the eigenvectors of the covariance matrix; the eigenvalues are the variances along them.

Relevant formulae. $C\mathbf{v} = \lambda\mathbf{v}$; explained fraction $= \lambda_1 / \sum_k \lambda_k = \lambda_1 / \operatorname{tr} C$.

Step-by-step explanation. PCA diagonalises the covariance matrix. $\det(C - \lambda I) = (2-\lambda)^2 - 1 = 0$ gives $\lambda = 3$ and $\lambda = 1$. For $\lambda = 3$: $(C - 3I)\mathbf{v} = 0 \Rightarrow -v_1 + v_2 = 0$, so $\mathbf{v}_1 = (1, 1)/\sqrt{2}$; for $\lambda = 1$, $\mathbf{v}_2 = (1, -1)/\sqrt{2}$. The total variance is $\operatorname{tr} C = 4 = 3 + 1$, so the first component explains $3/4 = 75\,\%$ along $(1, 1)/\sqrt{2}$.

Why the other options are incorrect.

(A) 50 % along $(1, 0)$ reads variance off the diagonal and ignores the covariance $C_{12} = 1$, which tilts the main axis to 45°.

(B) 67 % divides the diagonal entry 2 by $2 + 1$; the eigenvalues, not the entries, give the variances.

(D) $(1, -1)/\sqrt{2}$ is the eigenvector of the *smaller* eigenvalue $\lambda = 1$ (25 %).

Key takeaway. Diagonalise: $\lambda = 3, 1$; the first component carries $3/4$ of the variance along $(1, 1)/\sqrt{2}$.

Q.5 Single-correct MCQ · Bayes' theorem; precision at low prevalence

A screening classifier has recall (sensitivity) 90 % and false-positive rate 5 %. Only 1 % of the population is positive. A randomly chosen person is flagged positive. The probability that the person is really positive (the precision of the classifier on this population) is closest to

(A) 18 % **(B)** 90 % **(C)** 95 % **(D)** ✓ 15 %

Answer: (D) Detailed solution

Correct answer. (D) 15 %

Concepts required. Bayes' theorem converts $P(\text{flag} \mid \text{positive})$ into $P(\text{positive} \mid \text{flag})$; base rates matter.

Relevant formulae. $P(D \mid +) = \frac{P(+|D)P(D)}{P(+|D)P(D)+P(+|\bar{D})P(\bar{D})}$.

Step-by-step explanation. Bayes' theorem: $P(D \mid +) = \frac{P(+|D)P(D)}{P(+)}$ with $P(+) = P(+ \mid D)P(D) + P(+ \mid \bar{D})P(\bar{D}) = 0.9(0.01) + 0.05(0.99) = 0.009 + 0.0495 = 0.0585$. Hence $P(D \mid +) = 0.009/0.0585 = 0.154 \approx 15\,\%$. With rare positives most alarms are false alarms, which is why accuracy and recall alone can be misleading on imbalanced data.

Why the other options are incorrect.

(A) 18 % divides 0.009 by the false-positive rate 0.05 instead of by the total positive rate 0.0585.

(B) 90 % is the recall $P(+ \mid D)$, confused with $P(D \mid +)$ (the "prosecutor's fallacy").

(C) 95 % is the specificity $1 - 0.05$, a property of the test, not of a flagged person.

Key takeaway. With rare positives, even a good classifier has low precision: most alarms are false.

Q.6 Single-correct MCQ · Chain rule for one neuron

A single neuron computes $f = \sigma(wx + b)$ with the sigmoid σ, and the loss is $L = (f - y)^2$. For $x = 2$, $w = 0.5$, $b = -1$ and $y = 1$, the derivative $\partial L/\partial w$ equals

(A) −0.25 **(B)** +0.5 **(C)** −2 **(D)** ✓ −0.5

Answer: (D) Detailed solution

Correct answer. (D) −0.5

Concepts required. Back-propagation is the chain rule: multiply the local derivatives along the path from the loss to the parameter.

Relevant formulae. $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial f}\frac{\partial f}{\partial z}\frac{\partial z}{\partial w}$, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, $\sigma(0) = \frac{1}{2}$.

Step-by-step explanation. Forward pass: $z = wx + b = 0.5(2) - 1 = 0$, $f = \sigma(0) = 0.5$. Chain rule: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial f}\frac{\partial f}{\partial z}\frac{\partial z}{\partial w} = 2(f - y) \cdot \sigma(z)(1 - \sigma(z)) \cdot x = 2(-0.5)(0.25)(2) = -0.5$. The negative sign means that increasing w lowers the loss (it pushes f towards $y = 1$).

Why the other options are incorrect.

(A) −0.25 forgets the factor $\partial z/\partial w = x = 2$.

(B) +0.5 has the wrong sign: $f - y = -0.5 < 0$.

(C) −2 forgets the sigmoid derivative $\sigma'(0) = 0.25$.

Key takeaway. Write the computation graph and multiply: $2(f - y) \cdot \sigma'(z) \cdot x$.

Q.7 SINGLE-CORRECT MCQ · Softmax and cross-entropy

A three-class network outputs the logits $(z_1, z_2, z_3) = (2, 1, 0)$ and uses the softmax $p_k = e^{z_k}/\sum_j e^{z_j}$. The probability of class 1, and the cross-entropy loss $-\ln p_{\text{true}}$ if the true class is class 2, are approximately

(A) 0.245 and 1.408 **(B)** ✓ 0.665 and 1.408 **(C)** 0.667 and 1.099 **(D)** 0.665 and 0.408

Answer: (B) Detailed solution

Correct answer. **(B)** 0.665 and 1.408

Concepts required. Softmax turns logits into probabilities; cross-entropy is minus the log-probability of the true class.

Relevant formulae. $p_k = e^{z_k}/\sum_j e^{z_j}$; $\ell = -\ln p_{\text{true}} = -z_{\text{true}} + \ln \sum_j e^{z_j}$.

Step-by-step explanation. $\sum_j e^{z_j} = e^2 + e + 1 = 7.389 + 2.718 + 1 = 11.107$. Then $p_1 = 7.389/11.107 = 0.665$ and $p_2 = 2.718/11.107 = 0.245$. The loss uses the probability of the *true* class (class 2): $-\ln p_2 = \ln 11.107 - 1 = 2.408 - 1 = 1.408$.

Why the other options are incorrect.

(A) 0.245 is p_2, not the requested p_1.

(C) 0.667 and 1.099 normalise the logits linearly (2/3, 1/3) instead of exponentially.

(D) $0.408 = -\ln p_1$ is the loss if the true class were class 1.

Key takeaway. Exponentiate, normalise, then take $-\ln$ of the true class's probability.

Q.8 SINGLE-CORRECT MCQ · Parameter count of a dense network

A fully connected network has 8 inputs, one hidden layer of 4 neurons and 1 output neuron; every neuron has a bias. The number of trainable parameters is (compare: the 8-qubit QCNN of `QML.ipynb` has 12)

(A) 36 **(B)** ✓ 41 **(C)** 37 **(D)** 13

Answer: (B) Detailed solution

Correct answer. **(B)** 41

Concepts required. Counting parameters of dense layers: one weight per connection plus one bias per neuron.

Relevant formulae. Dense layer $n_{\text{in}} \to n_{\text{out}}$: $n_{\text{in}} n_{\text{out}} + n_{\text{out}}$ parameters.

Step-by-step explanation. A dense layer from n_{in} to n_{out} neurons has $n_{\text{in}} n_{\text{out}}$ weights and n_{out} biases. Input $\to$ hidden: $8 \times 4 + 4 = 36$. Hidden $\to$ output: $4 \times 1 + 1 = 5$. Total $36 + 5 = 41$. (The QCNN needs only 12 because each of its 3 layers *shares* 4 parameters among all its blocks, see Section C.)

Why the other options are incorrect.

(A) 36 counts only the weights $32 + 4$, without biases.

(C) 37 forgets the four hidden biases $(32 + 4 + 1)$.

(D) 13 counts neurons $(8 + 4 + 1)$, not parameters.

Key takeaway. $8 \cdot 4 + 4 + 4 \cdot 1 + 1 = 41$; weight sharing (as in CNNs and the QCNN) is what keeps parameter counts small.

Q.9 SINGLE-CORRECT MCQ · Gaussian (RBF) kernel

The Gaussian (RBF) kernel is $k(\mathbf{u},\mathbf{v}) = \exp(-\|\mathbf{u}-\mathbf{v}\|^2/(2\sigma^2))$. For $\mathbf{u}=(1,2,2)$, $\mathbf{v}=(2,0,1)$ and $\sigma=1$, $k(\mathbf{u},\mathbf{v})$ equals

(A) e^{-6} **(B)** $e^{-\sqrt{6}/2}$ **(C)** $e^{-3/2}$ **(D)** ✓ e^{-3}

Answer: (D) Detailed solution

Correct answer. (D) e^{-3}

Concepts required. The RBF kernel is a similarity measure that decays with the squared Euclidean distance.

Relevant formulae. $k(\mathbf{u},\mathbf{v}) = e^{-\|\mathbf{u}-\mathbf{v}\|^2/(2\sigma^2)}$, $\|\mathbf{a}\|^2 = \sum_i a_i^2$.

Step-by-step explanation. $\mathbf{u}-\mathbf{v} = (-1,2,1)$, so $\|\mathbf{u}-\mathbf{v}\|^2 = 1+4+1 = 6$ (the *squared* norm). With $2\sigma^2 = 2$: $k = e^{-6/2} = e^{-3} \approx 0.050$. Kernels measure similarity: $k=1$ for identical points and $k \to 0$ for distant ones. Quantum kernels (Section C) replace this formula by a state overlap.

Why the other options are incorrect.

(A) e^{-6} forgets the denominator $2\sigma^2 = 2$.

(B) $e^{-\sqrt{6}/2}$ uses the distance $\sqrt{6}$ instead of the squared distance.

(C) $e^{-3/2}$ divides by 4 instead of $2\sigma^2 = 2$.

Key takeaway. Square the distance, divide by $2\sigma^2$, exponentiate the negative.

Q.10 SINGLE-CORRECT MCQ · Kronecker product

For $A = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$ and $B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ the Kronecker (tensor) product $A \otimes B$ is

(A) $\begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 1 \\ 2 & 0 & 1 & 0 \end{pmatrix}$ **(B)** $\begin{pmatrix} 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \\ 1 & 2 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$ **(C)** $\begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 2 \\ 1 & 0 & 2 & 0 \end{pmatrix}$ **(D)** ✓ $\begin{pmatrix} 0 & 1 & 0 & 2 \\ 1 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$

Answer: (D) Detailed solution

Correct answer. (D) $\begin{pmatrix} 0 & 1 & 0 & 2 \\ 1 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$

Concepts required. The Kronecker product replaces every entry a_{ij} of A by the block $a_{ij}B$; it is how multi-qubit operators are built.

Relevant formulae. $A \otimes B = (a_{ij}B)_{ij}$; $(A\otimes B)(C\otimes D) = AC \otimes BD$; $\operatorname{tr}(A\otimes B) = \operatorname{tr}A \operatorname{tr}B$.

Step-by-step explanation. By definition $A \otimes B = \begin{pmatrix} a_{11}B & a_{12}B \\ a_{21}B & a_{22}B \end{pmatrix} = \begin{pmatrix} 1\cdot B & 2\cdot B \\ 0\cdot B & 1\cdot B \end{pmatrix}$: the top-left block is B, the top-right block is $2B$, the bottom-left block is zero and the bottom-right block is B. Writing the blocks out gives the first matrix. (The second matrix is $B\otimes A$: the Kronecker product is not commutative. Check: $\operatorname{tr}(A\otimes B) = \operatorname{tr}A \operatorname{tr}B = 2\cdot 0 = 0$.)

Why the other options are incorrect.

(A) This is $A^{\mathsf{T}} \otimes B = (A\otimes B)^{\mathsf{T}}$ (as B is symmetric): the block $2B$ sits bottom-left instead of top-right.

(B) This is $B \otimes A$ (blocks $b_{ij}A$); the Kronecker product is not commutative.

(C) This is $A\otimes B$ with its two block rows exchanged, $(X\otimes I)(A\otimes B) = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 2 \end{smallmatrix}\right) \otimes B$: the zero block must be bottom-left.

Key takeaway. Top-left block $a_{11}B$, top-right $a_{12}B$, …: order matters, $A\otimes B \neq B \otimes A$.

Q.11 Single-correct MCQ · Standard error of an accuracy

A classifier is correct on 17 of 20 test images (accuracy $\hat{a} = 0.85$, the default test-set size and the recorded test accuracy of `QML.ipynb`). Treating the images as independent trials, the standard error of the accuracy estimate is about

(A) 1.8 percentage points
(B) ✓ 8 percentage points
(C) 36 percentage points
(D) 0.6 percentage points

Answer: (B) Detailed solution

Correct answer. **(B)** 8 percentage points
Concepts required. An accuracy measured on M independent test examples is a binomial proportion with sampling error.
Relevant formulae. $\mathrm{SE}(\hat{a}) = \sqrt{\hat{a}(1-\hat{a})/M}$; 95 % interval $\approx \hat{a} \pm 2\,\mathrm{SE}$.
Step-by-step explanation. The number of correct answers is binomial, so the estimated accuracy has standard error $\mathrm{SE} = \sqrt{\hat{a}(1-\hat{a})/M} = \sqrt{0.85 \times 0.15/20} = \sqrt{0.006375} = 0.080$, i.e. about 8 percentage points. A rough 95 % interval is 85 ± 16 %: twenty test images are far too few to distinguish models whose accuracies differ by a few per cent.
Why the other options are incorrect.
(A) 1.8 points divides by $M^2 = 400$ inside the square root.
(C) 36 points is $\sqrt{0.85 \cdot 0.15}$, the spread of a *single* trial (forgets dividing by M).
(D) 0.6 points is the *variance* $0.85 \cdot 0.15/20 = 0.0064$, not its square root.
Key takeaway. Twenty test images give ±8 points: never over-interpret small differences on small test sets.

Q.12 Single-correct MCQ · Data leakage and test-set tuning

A student standardises all 200 images of a dataset (mean and standard deviation of every pixel computed over all 200 images), then splits them into 150 training and 50 test images, picks the learning rate that gives the best *test* accuracy, and reports that test accuracy. Which statement is correct?

(A) The bias disappears if the split is changed from 150/50 to 100/100.
(B) ✓ The reported accuracy is optimistically biased: test data influenced both the preprocessing statistics and the choice of learning rate. Fit the scaler on the training data only, tune on a validation split and evaluate on the test set once.
(C) Only the standardisation is a problem; choosing the learning rate on the test set is legitimate because the weights are fitted on the training set only.
(D) The procedure is correct, because standardisation is a linear map that never uses the labels.

Answer: (B) Detailed solution

Correct answer. **(B)** The reported accuracy is optimistically biased: test data influenced both the preprocessing statistics and the choice of learning rate. Fit the scaler on the training data only, tune on a validation split and evaluate on the test set once.
Concepts required. The test set must be independent of every modelling decision (preprocessing and hyper-parameters); otherwise the estimate is biased.
Relevant formulae. Pipeline: split → fit preprocessing on train → tune on validation (or cross-validation) → evaluate on test once.
Step-by-step explanation. The test set must play no role in any decision about the model. Here it was used twice: (1) *data leakage in preprocessing* — the mean and standard deviation contain information about the test images; (2) *selection on the test set* — choosing the hyper-parameter with the best test score and then reporting that score is an optimistic (maximum-of-noisy-values)

estimate. The correct pipeline is: split first; fit every preprocessing step on the training split; choose hyper-parameters on a separate validation split (or by cross-validation); report the test score once. The size of the split does not remove either bias.

Why the other options are incorrect.

(A) The split ratio changes the variance of the estimate, not the bias caused by leakage and test-set tuning.

(C) Choosing a hyper-parameter by its test score and reporting that score is selection on the test set: an optimistically biased maximum.

(D) Not using labels is not enough: test-set statistics still leak into the model, and the learning rate was chosen on the test set.

Key takeaway. Touch the test set once, at the very end; everything learned from data is learned from the training split.

Q.13 SINGLE-CORRECT MCQ · Gradient of the MSE (matrix form)

For linear regression with design matrix $X \in \mathbb{R}^{M\times d}$, targets $\mathbf{y} \in \mathbb{R}^M$ and loss $L(\mathbf{w}) = \frac{1}{M}\|X\mathbf{w} - \mathbf{y}\|^2$, the gradient $\nabla_{\mathbf{w}} L$ is

(A) $\frac{2}{M}X^\mathsf{T}X\mathbf{w}$ **(B)** $\frac{2}{M}X(X\mathbf{w}-\mathbf{y})$

(C) ✓ $\frac{2}{M}X^\mathsf{T}(X\mathbf{w}-\mathbf{y})$ **(D)** $\frac{1}{M}X^\mathsf{T}(X\mathbf{w}-\mathbf{y})$

Answer: (C) Detailed solution

Correct answer. (C) $\frac{2}{M}X^\mathsf{T}(X\mathbf{w}-\mathbf{y})$

Concepts required. Matrix calculus for quadratic forms; the gradient must have the shape of $\mathbf{w}$ (length d).

Relevant formulae. $\nabla_{\mathbf{w}}\mathbf{w}^\mathsf{T}A\mathbf{w} = 2A\mathbf{w}$ (A symmetric), $\nabla_{\mathbf{w}}\mathbf{c}^\mathsf{T}\mathbf{w} = \mathbf{c}$.

Step-by-step explanation. Write $L = \frac{1}{M}(X\mathbf{w}-\mathbf{y})^\mathsf{T}(X\mathbf{w}-\mathbf{y}) = \frac{1}{M}(\mathbf{w}^\mathsf{T}X^\mathsf{T}X\mathbf{w} - 2\mathbf{y}^\mathsf{T}X\mathbf{w} + \mathbf{y}^\mathsf{T}\mathbf{y})$. Using $\nabla_{\mathbf{w}}(\mathbf{w}^\mathsf{T}A\mathbf{w}) = 2A\mathbf{w}$ for symmetric A and $\nabla_{\mathbf{w}}(\mathbf{c}^\mathsf{T}\mathbf{w}) = \mathbf{c}$: $\nabla L = \frac{1}{M}(2X^\mathsf{T}X\mathbf{w} - 2X^\mathsf{T}\mathbf{y}) = \frac{2}{M}X^\mathsf{T}(X\mathbf{w}-\mathbf{y})$, a vector of length d as required. Setting it to zero gives the normal equations $X^\mathsf{T}X\mathbf{w} = X^\mathsf{T}\mathbf{y}$.

Why the other options are incorrect.

(A) $\frac{2}{M}X^\mathsf{T}X\mathbf{w}$ drops the target term $-\frac{2}{M}X^\mathsf{T}\mathbf{y}$.

(B) $X(X\mathbf{w}-\mathbf{y})$ is not even defined in general (X is $M\times d$, the residual has length M).

(D) $\frac{1}{M}X^\mathsf{T}(X\mathbf{w}-\mathbf{y})$ forgets the factor 2 from differentiating a square.

Key takeaway. $\nabla L = \frac{2}{M}X^\mathsf{T}(\text{residual})$; setting it to zero gives the normal equations.

Q.14 SINGLE-CORRECT MCQ · Momentum

Gradient descent with (heavy-ball) momentum uses $v_t = \beta v_{t-1} + g_t$ and $w_t = w_{t-1} - \eta v_t$, with $v_0 = 0$. If the gradient is constant, $g_t = 1$, with $\beta = 0.9$ and $\eta = 0.1$, the total decrease $w_0 - w_3$ after three steps is

(A) ✓ 0.561 **(B)** 0.271 **(C)** 0.300 **(D)** 0.0561

Answer: (A) Detailed solution

Correct answer. (A) 0.561

Concepts required. Momentum accumulates past gradients in a velocity, accelerating motion along consistent directions.

Relevant formulae. $v_t = \beta v_{t-1} + g_t$, $w_t = w_{t-1} - \eta v_t$; with constant g, $v_t = g\frac{1-\beta^t}{1-\beta}$.

Step-by-step explanation. $v_1 = 0.9(0) + 1 = 1$, $v_2 = 0.9(1) + 1 = 1.9$, $v_3 = 0.9(1.9) + 1 = 2.71$. Total decrease $= \eta(v_1 + v_2 + v_3) = 0.1(1 + 1.9 + 2.71) = 0.1(5.61) = 0.561$. Without momentum

it would be $3\eta = 0.3$: momentum accelerates motion along directions where the gradient keeps the same sign (the velocity approaches $g/(1-\beta) = 10$).

Why the other options are incorrect.

(B) 0.271 counts only the last step ηv_3.

(C) 0.300 is plain gradient descent ($3\eta g$), ignoring the momentum term.

(D) 0.0561 uses the exponential-moving-average form $v_t = \beta v_{t-1} + (1-\beta) g_t$, which is a different update rule.

Key takeaway. Momentum: $v = 1, 1.9, 2.71$; the steps add up to 0.561, almost twice plain GD.

Q.15 MATCHING LISTS · Derivatives of activations

Match each activation function (List-I) with its derivative (List-II).

List-I		List-II	
(P)	sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$	**(1)**	$\sigma(z)$
(Q)	$\tanh z$	**(2)**	$1 - \tanh^2 z$
(R)	$\mathrm{ReLU}(z) = \max(0, z)$	**(3)**	$\sigma(z)(1-\sigma(z))$
(S)	softplus $\ln(1+e^z)$	**(4)**	1 for $z > 0$, 0 for $z < 0$

(A) ✓ P→3, Q→2, R→4, S→1 **(B)** P→2, Q→3, R→4, S→1

(C) P→1, Q→2, R→4, S→3 **(D)** P→3, Q→4, R→2, S→1

Answer: (A) **Detailed solution**

Correct answer. (A) P→3, Q→2, R→4, S→1

Concepts required. Derivatives of common activation functions, used in every back-propagation step.

Relevant formulae. $\sigma' = \sigma(1-\sigma)$, $\tanh' = 1 - \tanh^2$, $\mathrm{ReLU}' = [z > 0]$, $\frac{d}{dz}\ln(1+e^z) = \sigma(z)$.

Step-by-step explanation.

- Sigmoid: $\sigma' = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z) \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(1-\sigma)$ (3).
- tanh: $\frac{d}{dz}\frac{\sinh z}{\cosh z} = \frac{\cosh^2 z - \sinh^2 z}{\cosh^2 z} = 1 - \tanh^2 z$ (2).
- ReLU: slope 1 for $z > 0$, 0 for $z < 0$ (not differentiable at 0; frameworks use 0 or 1 there) (4).
- Softplus: $\frac{d}{dz}\ln(1+e^z) = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}} = \sigma(z)$ (1).

Hence P→3, Q→2, R→4, S→1.

Why the other options are incorrect.

(B) P→2, Q→3 swaps the derivatives of sigmoid and tanh; they are related ($\tanh z = 2\sigma(2z) - 1$) but not equal.

(C) P→1, S→3 swaps sigmoid and softplus: the derivative of the softplus *is* the sigmoid, the derivative of the sigmoid is $\sigma(1-\sigma)$.

(D) Q→4, R→2 gives tanh a step derivative and ReLU the derivative $1 - \tanh^2$: tanh is smooth, ReLU is piecewise linear.

Key takeaway. Sigmoid → $\sigma(1-\sigma)$, tanh → $1 - \tanh^2$, ReLU → step, softplus → sigmoid.

Q.16 MATCHING LISTS · Over-/underfitting, early stopping, imbalance

Match each observation (List-I) with the correct diagnosis and remedy (List-II).

List-I		List-II	
(P)	training accuracy 99 %, test accuracy 70 %	**(1)**	underfitting (high bias): increase model capacity or improve the features
(Q)	training and test accuracy both 55 % on a balanced two-class task	**(2)**	accuracy is misleading: report precision, recall and F1
(R)	validation loss is lowest at epoch 40 and rises afterwards while training loss keeps falling	**(3)**	overfitting (high variance): regularise, add data or simplify the model
(S)	a model that predicts "healthy" for everybody has 99 % accuracy when 1 % of patients are sick	**(4)**	stop at the validation minimum (early stopping)

(A) P→4, Q→1, R→3, S→2
(B) ✓ P→3, Q→1, R→4, S→2
(C) P→1, Q→3, R→4, S→2
(D) P→3, Q→1, R→2, S→4

Answer: (B) Detailed solution

Correct answer. (B) P→3, Q→1, R→4, S→2
Concepts required. Diagnosing models from training/validation/test behaviour; choosing metrics for imbalanced data.
Relevant formulae. Generalisation gap = test error − training error; high bias: both errors high; high variance: large gap.
Step-by-step explanation.

- P: a large gap between training and test performance is the signature of *overfitting* (3).
- Q: both errors high and close to chance level (50 %): the model is too simple — *underfitting* (1).
- R: the classic early-stopping picture: keep the parameters at the validation minimum (4).
- S: with 1 % positives the trivial classifier reaches 99 % accuracy but recall 0: use precision, recall, F1 (2).

Hence P→3, Q→1, R→4, S→2.
Why the other options are incorrect.

(A) P→4, R→3: early stopping needs a validation curve; P only gives final accuracies, and R is the textbook early-stopping case.

(C) P→1, Q→3 swaps overfitting and underfitting: a 29-point gap is high variance, 55 %/55 % is high bias.

(D) R→2, S→4 assigns early stopping to the imbalance problem; the rising validation loss is the early-stopping signal.

Key takeaway. Gap ⇒ overfitting; both low ⇒ underfitting; rising validation loss ⇒ stop early; imbalance ⇒ P/R/F1.

Q.17 Assertion–Reason · XOR and linear separability

Assertion (A): No linear classifier $\hat{y} = \text{sign}(w_1 x_1 + w_2 x_2 + b)$ can classify all four XOR points $(0,0) \mapsto -1$, $(1,1) \mapsto -1$, $(0,1) \mapsto +1$, $(1,0) \mapsto +1$ correctly.

Reason (R): The two classes of the XOR data cannot be separated by a straight line.

(A) ✓ Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A

(B) Both A and R are true, but R is *not* the correct explanation of A

(C) A is true, but R is false

(D) A is false, but R is true

Answer: (A) Detailed solution

Correct answer. **(A)** Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A

Concepts required. Linear separability: a linear classifier can only realise a half-plane decision boundary.

Relevant formulae. $\hat{y} = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$; summing the constraints of the two classes gives a contradiction for XOR.

Step-by-step explanation. **R is true:** suppose $g(\mathbf{x}) = w_1 x_1 + w_2 x_2 + b$ separated them. Then $g(0,1) > 0$ and $g(1,0) > 0$ give $w_1 + w_2 + 2b > 0$, while $g(0,0) < 0$ and $g(1,1) < 0$ give $w_1 + w_2 + 2b < 0$ — a contradiction. **A is true**, and it is exactly the statement that a linear decision boundary does not exist, so **R explains A**. A hidden layer (or a feature map such as $x_1 x_2$) makes XOR separable — the same idea motivates quantum feature maps. Answer: (A).

Why the other options are incorrect.

(B) needs R not to explain A, but non-separability is exactly why no linear classifier works.

(C) needs R false, but XOR is the standard example of non-separable data (the inequalities contradict each other).

(D) needs A false, but no choice of w_1, w_2, b classifies all four points.

Key takeaway. XOR needs a non-linear feature (hidden layer, kernel, feature map) — the motivation for feature maps in QML.

Q.18 Statement I / II · GD step-size limit; effect of L_2 penalty

Statement I: For $L(\mathbf{w}) = \frac{1}{2}\mathbf{w}^\mathsf{T} H \mathbf{w}$ with H symmetric positive definite and largest eigenvalue $\lambda_{\max}$, gradient descent with constant step η converges to $\mathbf{0}$ from every starting point if and only if $0 < \eta < 2/\lambda_{\max}$.

Statement II: Adding the penalty $\lambda\|\mathbf{w}\|^2$ $(\lambda > 0)$ to the training loss of linear regression can never increase the training mean squared error of the fitted model.

(A) Both Statement I and Statement II are true

(B) Both Statement I and Statement II are false

(C) ✓ Statement I is true, but Statement II is false

(D) Statement I is false, but Statement II is true

Answer: (C) Detailed solution

Correct answer. **(C)** Statement I is true, but Statement II is false

Concepts required. Convergence of gradient descent on quadratics; the effect of regularisation on training error.

Relevant formulae. $\mathbf{w}_{t+1} = (I - \eta H)\mathbf{w}_t$ converges iff $|1 - \eta\lambda_i| < 1$ $\forall i$; ridge: $\min \|X\mathbf{w} - \mathbf{y}\|^2 + \lambda\|\mathbf{w}\|^2$.

Step-by-step explanation. **I is true:** $\nabla L = H\mathbf{w}$, so $\mathbf{w}_{t+1} = (I - \eta H)\mathbf{w}_t$. In the eigenbasis of

H each component is multiplied by $1-\eta\lambda_i$ per step; it shrinks to 0 for every start iff $|1-\eta\lambda_i|<1$, i.e. $0<\eta<2/\lambda_i$, for all i, i.e. $0<\eta<2/\lambda_{\max}$. **II is false:** the unregularised least-squares solution *minimises* the training MSE, so any other solution — in particular the ridge solution — has a training MSE that is *at least as large* (strictly larger unless $X^{\mathsf{T}}\mathbf{y}=0$). Regularisation trades a higher training error for lower variance and, ideally, a lower test error. Answer: (C).

Why the other options are incorrect.

(A) needs II true, but the unregularised solution minimises the training MSE, so a penalty cannot lower it.

(B) needs I false, but the eigen-analysis gives exactly the condition $0<\eta<2/\lambda_{\max}$.

(D) needs I false and II true; both judgements are reversed.

Key takeaway. Step size limited by curvature ($\eta<2/\lambda_{\max}$); regularisation raises training error to lower variance.

Paragraph for Questions 19 and 20

A tiny neural network has two inputs, one hidden layer with two ReLU neurons and one sigmoid output neuron:

$$\mathbf{h}=\mathrm{ReLU}(W_1\mathbf{x}+\mathbf{b}_1),\qquad p=\sigma(\mathbf{w}_2\cdot\mathbf{h}+b_2),\qquad \sigma(z)=\frac{1}{1+e^{-z}},\qquad \mathrm{ReLU}(a)=\max(0,a),$$

with $W_1=\begin{pmatrix}1 & -1\\ 2 & 1\end{pmatrix}$ (row j = weights into hidden neuron j), $\mathbf{b}_1=(0,-2)$, $\mathbf{w}_2=(0.5,-1)$ and $b_2=-1$. It is trained with the binary cross-entropy $L=-\big[y\ln p+(1-y)\ln(1-p)\big]$. Consider the single training example $\mathbf{x}=(1,-1)$ with label $y=1$.

Q.19 PARAGRAPH BASED · Forward pass of a small network

For the example $\mathbf{x}=(1,-1)$, $y=1$, the network output p and the loss L are

(A) ✓ $p=0.5$, $L=\ln 2\approx 0.693$ **(B)** $p=0.5$, $L=0.25$

(C) $p\approx 0.269$, $L\approx 1.313$ **(D)** $p\approx 0.731$, $L\approx 0.313$

Answer: (A) **Detailed solution**

Correct answer. (A) $p=0.5$, $L=\ln 2\approx 0.693$

Concepts required. Forward pass through a small network: affine map, ReLU, affine map, sigmoid, then the loss.

Relevant formulae. $\mathbf{a}=W_1\mathbf{x}+\mathbf{b}_1$, $\mathbf{h}=\max(0,\mathbf{a})$, $z=\mathbf{w}_2\cdot\mathbf{h}+b_2$, $p=\sigma(z)$, $L=-\ln p$ for $y=1$.

Step-by-step explanation. Pre-activations: $W_1\mathbf{x}+\mathbf{b}_1=(1\cdot 1+(-1)(-1),\ 2\cdot 1+1\cdot(-1))+(0,-2)=(2,1)+(0,-2)=(2,-1)$. ReLU: $\mathbf{h}=(2,0)$ (the second neuron is inactive). Output: $z=0.5(2)+(-1)(0)-1=0$, so $p=\sigma(0)=0.5$. Loss with $y=1$: $L=-\ln p=\ln 2\approx 0.693$.

Why the other options are incorrect.

(B) $L=0.25$ is the squared error $(p-y)^2$, not the cross-entropy.

(C) $p=0.269=\sigma(-1)$ flips the sign of z (e.g. $b_2=-1$ counted twice).

(D) $p=0.731$ forgets the ReLU: it uses $h_2=-1$ instead of 0, so $z=1$.

Key takeaway. Apply the ReLU before the next layer: $\mathbf{h}=(2,0)$, $z=0$, $p=\frac{1}{2}$, $L=\ln 2$.

Q.20 PARAGRAPH BASED · Back-propagation; dead ReLU

Back-propagation for the same example gives $\partial L/\partial b_2$ and the gradient of L with respect to the *second row* of W_1 (the weights into hidden neuron 2) equal to

(A) ✓ -0.5 and $(0, 0)$
(B) -0.125 and $(0, 0)$
(C) -0.5 and $(0.5, -0.5)$
(D) $+0.5$ and $(0, 0)$

Answer: (A) Detailed solution

Correct answer. **(A)** -0.5 and $(0, 0)$
Concepts required. Backward pass: the sigmoid–cross-entropy pair gives $\partial L/\partial z = p - y$; ReLU passes gradient only where it is active.
Relevant formulae. $\frac{\partial L}{\partial z} = p - y$, $\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z}$, $\frac{\partial L}{\partial a_j} = \frac{\partial L}{\partial z} w_{2,j}[a_j > 0]$, $\frac{\partial L}{\partial W_{1,j\cdot}} = \frac{\partial L}{\partial a_j}\mathbf{x}$.
Step-by-step explanation. For sigmoid + binary cross-entropy, $\frac{\partial L}{\partial z} = p - y = 0.5 - 1 = -0.5$ (the factor σ' cancels). Since $z = \mathbf{w}_2 \cdot \mathbf{h} + b_2$: $\frac{\partial L}{\partial b_2} = -0.5$ and $\frac{\partial L}{\partial \mathbf{w}_2} = -0.5\,\mathbf{h} = (-1, 0)$. Hidden neuron 2: $\frac{\partial L}{\partial a_2} = \frac{\partial L}{\partial z}\, w_{2,2}\, \mathrm{ReLU}'(a_2) = (-0.5)(-1) \cdot 0 = 0$, because $a_2 = -1 < 0$. Therefore $\frac{\partial L}{\partial W_{1,(2,\cdot)}} = 0 \cdot \mathbf{x} = (0, 0)$: an inactive ("dead") ReLU passes no gradient. (Neuron 1: $\frac{\partial L}{\partial a_1} = (-0.5)(0.5)(1) = -0.25$, row gradient $(-0.25, 0.25)$.)
Why the other options are incorrect.
(B) -0.125 multiplies by $\sigma'(0) = 0.25$, which cancels for sigmoid + cross-entropy.
(C) $(0.5, -0.5)$ ignores the ReLU derivative, which is 0 because $a_2 = -1 < 0$.
(D) $+0.5$ has the wrong sign: $p - y = -0.5$.
Key takeaway. Inactive ReLUs receive zero gradient ("dead neurons"); for sigmoid + BCE the output error is simply $p - y$.

SILICO FELLER

SECTION B · Quantum Computing Foundations Q.21–40

Q.21 Single-correct MCQ · Expectation values from amplitudes

For the qubit state $|\psi\rangle = \frac{1}{2}|0\rangle + \frac{\sqrt{3}}{2}e^{i\pi/4}|1\rangle$ the expectation values $(\langle X\rangle, \langle Z\rangle)$ are

(A) $(\frac{\sqrt{6}}{2}, -\frac{1}{2})$ **(B)** ✓ $(\frac{\sqrt{6}}{4}, -\frac{1}{2})$ **(C)** $(\frac{\sqrt{6}}{4}, \frac{1}{2})$ **(D)** $(\frac{\sqrt{3}}{2}, -\frac{1}{2})$

Answer: (B) Detailed solution

Correct answer. (B) $(\frac{\sqrt{6}}{4}, -\frac{1}{2})$
Concepts required. Born rule and expectation values of Pauli observables for a single qubit.
Relevant formulae. $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$: $\langle Z\rangle = |\alpha|^2 - |\beta|^2$, $\langle X\rangle = 2\,\mathrm{Re}(\alpha^*\beta)$, $\langle Y\rangle = 2\,\mathrm{Im}(\alpha^*\beta)$.
Step-by-step explanation. For $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$: $\langle Z\rangle = |\alpha|^2 - |\beta|^2$ and $\langle X\rangle = 2\,\mathrm{Re}(\alpha^*\beta)$. Here $|\alpha|^2 = \frac{1}{4}$, $|\beta|^2 = \frac{3}{4}$, so $\langle Z\rangle = -\frac{1}{2}$. And $\langle X\rangle = 2 \cdot \frac{1}{2} \cdot \frac{\sqrt{3}}{2}\cos\frac{\pi}{4} = \frac{\sqrt{3}}{2} \cdot \frac{\sqrt{2}}{2} = \frac{\sqrt{6}}{4} \approx 0.612$. (Check: $\langle Y\rangle = 2\,\mathrm{Im}(\alpha^*\beta) = \frac{\sqrt{6}}{4}$ and $\langle X\rangle^2 + \langle Y\rangle^2 + \langle Z\rangle^2 = \frac{3}{8} + \frac{3}{8} + \frac{1}{4} = 1$: a pure state lies on the Bloch sphere.)
Why the other options are incorrect.
(A) $\frac{\sqrt{6}}{2} > 1$ is impossible for an expectation value of X (forgot the factor $\frac{1}{2}$ in α).
(C) $\langle Z\rangle = +\frac{1}{2}$ swaps $P(0)$ and $P(1)$; $|1\rangle$ is the more likely outcome.
(D) $\frac{\sqrt{3}}{2}$ ignores the relative phase $e^{i\pi/4}$, which reduces $\langle X\rangle$ by $\cos\frac{\pi}{4}$.
Key takeaway. Relative phases change $\langle X\rangle$, $\langle Y\rangle$ but not $\langle Z\rangle$.

Q.22 Single-correct MCQ · Pauli algebra

For real numbers a, b, c the operator $(aX + bY + cZ)^2$ equals

(A) $(a^2+b^2+c^2)\,I + 2i(abZ + bcX + caY)$ **(B)** ✓ $(a^2+b^2+c^2)\,I$
(C) $a^2X + b^2Y + c^2Z$ **(D)** $(a+b+c)^2\,I$

Answer: (B) Detailed solution

Correct answer. (B) $(a^2+b^2+c^2)\,I$
Concepts required. Pauli matrices square to the identity and anticommute pairwise.
Relevant formulae. $X^2 = Y^2 = Z^2 = I$; $XY = -YX = iZ$, $YZ = -ZY = iX$, $ZX = -XZ = iY$.
Step-by-step explanation. Expand: $a^2X^2 + b^2Y^2 + c^2Z^2 + ab(XY + YX) + bc(YZ + ZY) + ca(ZX + XZ)$. Every Pauli matrix squares to I, and different Pauli matrices *anticommute*: $XY = iZ = -YX$, $YZ = iX = -ZY$, $ZX = iY = -XZ$, so all cross terms vanish. Result: $(a^2+b^2+c^2)I$. Consequence: $\hat{n}\cdot\vec{\sigma}$ with a unit vector $\hat{n}$ has eigenvalues ± 1 — every such observable is a "rotated Z".
Why the other options are incorrect.
(A) The extra term $2i(abZ + \dots)$ assumes $XY = YX$; anticommutation makes $XY + YX = 0$.
(C) $a^2X + b^2Y + c^2Z$ forgets that $X^2 = I$ (not X).
(D) $(a+b+c)^2 I$ treats the Paulis as commuting numbers; cross terms cancel instead.
Key takeaway. $(\hat{n}\cdot\vec{\sigma})^2 = |\hat{n}|^2 I$: Pauli combinations behave like rotated Z operators.

SHIGO FELLER

Q.23 Single-correct MCQ · Period of $f(\theta)$; global phase

A circuit contains the parameter θ in exactly one gate $R_Y(\theta) = e^{-i\theta Y/2}$, and $f(\theta) = \langle\psi(\theta)|M|\psi(\theta)\rangle$ is the expectation value of an observable M measured at the end. Although $R_Y(\theta{+}2\pi) = -R_Y(\theta)$, the smallest period that f is guaranteed to have is

(A) 4π **(B)** π **(C)** $\pi/2$ **(D)** ✓ 2π

Answer: (D) Detailed solution

Correct answer. (D) 2π

Concepts required. Global phases do not affect expectation values; a single rotation gate produces a single-frequency trigonometric function.

Relevant formulae. $R_Y(\theta + 2\pi) = -R_Y(\theta)$; $f(\theta) = a + b\cos\theta + c\sin\theta$.

Step-by-step explanation. Write the circuit as $|\psi(\theta)\rangle = V R_Y(\theta) W |0\rangle$. Replacing θ by $\theta + 2\pi$ multiplies the state by -1, a *global* phase, and $f = \langle\psi|M|\psi\rangle$ contains $|\psi\rangle$ and $\langle\psi|$, so the sign cancels: $f(\theta + 2\pi) = f(\theta)$. In general $f(\theta) = a + b\cos\theta + c\sin\theta$ (only the frequency 1 appears), whose period is exactly 2π unless $b = c = 0$; π or $\pi/2$ would require extra symmetry. The operator itself has period 4π.

Why the other options are incorrect.

(A) 4π is the period of the *operator* $R_Y(\theta)$; the sign -1 is a global phase that cancels in $\langle\psi|M|\psi\rangle$.

(B) π would need $f(\theta + \pi) = f(\theta)$, but $\cos(\theta + \pi) = -\cos\theta$: the frequency-1 terms change sign.

(C) $\pi/2$ would need frequency 4, which a single R_Y cannot produce.

Key takeaway. One rotation gate ⇒ f is a sinusoid of period 2π in its angle.

Q.24 Single-correct MCQ · Product vs. entangled states

Which of the following two-qubit states is a *product* (non-entangled) state?

(A) $\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle)$ **(B)** $\frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$

(C) $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ **(D)** ✓ $\frac{1}{2}(|00\rangle - |01\rangle + |10\rangle - |11\rangle)$

Answer: (D) Detailed solution

Correct answer. (D) $\frac{1}{2}(|00\rangle - |01\rangle + |10\rangle - |11\rangle)$

Concepts required. A pure two-qubit state is a product state iff its 2×2 coefficient matrix has rank 1.

Relevant formulae. $|\psi\rangle = \sum_{ab} c_{ab}|ab\rangle$ is a product state $\Leftrightarrow c_{00}c_{11} - c_{01}c_{10} = 0$.

Step-by-step explanation. Write $|\psi\rangle = \sum c_{ab}|ab\rangle$ and arrange the coefficients as a 2×2 matrix $C = (c_{ab})$. A pure state is a product state iff $\det C = c_{00}c_{11} - c_{01}c_{10} = 0$ (rank 1). First state: $\frac{1}{2}\begin{pmatrix} 1 & -1 \\ 1 & -1 \end{pmatrix}$, determinant $\frac{1}{4}(-1 + 1) = 0$. Indeed it equals $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |+\rangle\,|-\rangle$. The other three have $\det C = -\frac{1}{2}$, $\frac{1}{2}$ and $\frac{1}{2}$: all entangled.

Why the other options are incorrect.

(A) $\frac{1}{2}(1, 1, 1, -1)$ has $c_{00}c_{11} - c_{01}c_{10} = \frac{1}{4}(-1 - 1) \neq 0$: entangled (it is CZ$|{+}{+}\rangle$).

(B) $\frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$ is the singlet, also maximally entangled.

(C) $|\Phi^+\rangle$ is the standard maximally entangled Bell state (determinant $\frac{1}{2}$).

Key takeaway. Check the determinant of the coefficient matrix: zero means product state.

Q.25 SINGLE-CORRECT MCQ · Bell-state correlations

For the Bell state $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ the expectation values $(\langle Z_0\rangle,\ \langle Z_0Z_1\rangle,\ \langle X_0X_1\rangle)$ are

(A) $(0,\ 0,\ 0)$ **(B)** $(1,\ 1,\ 1)$ **(C)** ✓ $(0,\ 1,\ 1)$ **(D)** $(0,\ 1,\ 0)$

Answer: (C) Detailed solution

Correct answer. (C) $(0,\ 1,\ 1)$
Concepts required. Bell states: random local outcomes, perfect correlations.
Relevant formulae. $Z_0Z_1\,|ab\rangle = (-1)^{a\oplus b}\,|ab\rangle$, $X_0X_1\,|ab\rangle = |\bar{a}\bar{b}\rangle$; $|\Phi^+\rangle$ is a $+1$ eigenstate of Z_0Z_1 and of X_0X_1.
Step-by-step explanation. $\langle Z_0\rangle$: outcomes 00 and 11 each with probability $\frac{1}{2}$, so qubit 0 gives $+1$ and -1 equally: 0. $\langle Z_0Z_1\rangle$: $Z_0Z_1\,|ab\rangle = (-1)^{a+b}\,|ab\rangle$; both 00 and 11 have even parity, so $+1$. $\langle X_0X_1\rangle$: $X\otimes X$ maps $|00\rangle \leftrightarrow |11\rangle$, so $X_0X_1\,|\Phi^+\rangle = |\Phi^+\rangle$: eigenvalue $+1$. The individual qubits look completely random, but their correlations are perfect — in two different bases.
Why the other options are incorrect.
(A) $(0,0,0)$ multiplies the marginals, $\langle Z_0\rangle\langle Z_1\rangle$; correlated qubits do not factorise.
(B) $\langle Z_0\rangle = 1$ would mean qubit 0 is always 0, but $|11\rangle$ occurs half of the time.
(D) $\langle X_0X_1\rangle = 0$ misses that $X\otimes X$ maps $|00\rangle \leftrightarrow |11\rangle$ and leaves $|\Phi^+\rangle$ unchanged.
Key takeaway. Entangled: single-qubit expectations 0, two-qubit correlations ± 1.

Q.26 SINGLE-CORRECT MCQ · Local rotation on an entangled qubit

Starting from $|00\rangle$, a circuit applies H to qubit 0, a CNOT (control 0, target 1) and then $R_Y(\theta)$ to qubit 1. The expectation value $\langle Z_1\rangle$ of the final state is

(A) $\cos\theta$ **(B)** $\cos^2(\theta/2)$ **(C)** ✓ 0 for every θ **(D)** $-\sin\theta$

Answer: (C) Detailed solution

Correct answer. (C) 0 for every θ
Concepts required. The reduced state of one half of a maximally entangled pair is $I/2$, which is invariant under local unitaries.
Relevant formulae. $\rho_1 = \mathrm{tr}_0\,|\Phi^+\rangle\langle\Phi^+| = \frac{I}{2}$; $\langle Z_1\rangle = \mathrm{tr}(\rho_1 U^\dagger Z U)$.
Step-by-step explanation. After H and CNOT the state is $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. The reduced state of qubit 1 is $\rho_1 = \mathrm{tr}_0\,|\Phi^+\rangle\langle\Phi^+| = \frac{1}{2}I$ (maximally mixed). A local unitary on qubit 1 maps it to $R_Y\,\frac{I}{2}\,R_Y^\dagger = \frac{I}{2}$, so $\langle Z_1\rangle = \mathrm{tr}(\rho_1 Z) = 0$ for every θ. A single-qubit readout of an entangled qubit can be completely insensitive to a local parameter (here $\langle Z_0Z_1\rangle = \cos\theta$ does depend on θ) — a small version of the trainability problems of Section D.
Why the other options are incorrect.
(A) $\cos\theta$ would be correct if qubit 1 started in $|0\rangle$; after the CNOT it is entangled (and $\langle Z_0Z_1\rangle = \cos\theta$).
(B) $\cos^2(\theta/2)$ is $P(0)$ for $R_Y(\theta)\,|0\rangle$, not an expectation value, and ignores the entanglement.
(D) $-\sin\theta$ is $\langle Z\rangle$ for $R_Y(\theta)\,|+\rangle$, a product-state calculation.
Key takeaway. Local readout of an entangled qubit can be blind to local parameters.

Q.27 SINGLE-CORRECT MCQ · $\langle Z\rangle$ from shots; standard error

A qubit is prepared and measured 1000 times in the computational basis; outcome 0 occurs 620 times. The estimate of $\langle Z\rangle$ and its standard error are approximately

(A) 0.62 ± 0.015 **(B)** ✓ 0.24 ± 0.031 **(C)** 0.24 ± 0.0009 **(D)** 0.24 ± 0.015

Answer: (B) Detailed solution

Correct answer. **(B)** 0.24 ± 0.031
Concepts required. Estimating expectation values from shots; binomial statistics.
Relevant formulae. $\widehat{\langle Z\rangle} = \hat{P}(0) - \hat{P}(1)$, $\mathrm{SE} = \sqrt{(1-\langle Z\rangle^2)/S}$.
Step-by-step explanation. $\widehat{\langle Z\rangle} = \hat{P}(0) - \hat{P}(1) = 0.62 - 0.38 = 0.24$. One shot gives ± 1 with variance $1-\langle Z\rangle^2$, so the mean of S shots has standard error $\sqrt{(1-\langle Z\rangle^2)/S} = \sqrt{(1-0.0576)/1000} = \sqrt{0.000942} = 0.0307$. (Equivalently $2\sqrt{\hat{p}(1-\hat{p})/S} = 2\sqrt{0.62\cdot 0.38/1000} = 2(0.01535)$.) Halving the error needs four times the shots.
Why the other options are incorrect.
(A) 0.62 is $\hat{P}(0)$, not $\langle Z\rangle$; and 0.015 is the error of $\hat{P}(0)$, half the error of $\langle Z\rangle$.
(C) 0.0009 is the *variance* $0.942/1000$, not the standard error.
(D) 0.015 forgets that $\langle Z\rangle = 2P(0) - 1$ doubles the error of $P(0)$.
Key takeaway. $\langle Z\rangle$ from S shots has error $\approx 1/\sqrt{S}$; 1000 shots give about ± 0.03.

Q.28 SINGLE-CORRECT MCQ · Tensor product; qubit ordering

The state $(R_Y(\pi/2) \otimes R_Y(\pi))\,|00\rangle$ (the first factor acts on qubit 0), written as the amplitude vector in the basis order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, is

(A) $\frac{1}{\sqrt{2}}(0,\ 0,\ 1,\ 1)$ **(B)** $\frac{1}{\sqrt{2}}(1,\ 0,\ 1,\ 0)$
(C) ✓ $\frac{1}{\sqrt{2}}(0,\ 1,\ 0,\ 1)$ **(D)** $\frac{1}{\sqrt{2}}(0,\ 1,\ 0,\ -1)$

Answer: (C) Detailed solution

Correct answer. **(C)** $\frac{1}{\sqrt{2}}(0,\ 1,\ 0,\ 1)$
Concepts required. Tensor products of single-qubit states and the basis ordering convention.
Relevant formulae. $R_Y(\theta)\,|0\rangle = \cos\frac{\theta}{2}\,|0\rangle + \sin\frac{\theta}{2}\,|1\rangle$; $|a\rangle \otimes |b\rangle = |ab\rangle$, index $= 2a + b$ (qubit 0 first).
Step-by-step explanation. $R_Y(\theta)\,|0\rangle = \cos\frac{\theta}{2}\,|0\rangle + \sin\frac{\theta}{2}\,|1\rangle$. Qubit 0: $R_Y(\pi/2)\,|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. Qubit 1: $R_Y(\pi)\,|0\rangle = |1\rangle$. The product is $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |1\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |11\rangle)$, i.e. the vector $\frac{1}{\sqrt{2}}(0,1,0,1)$ (qubit 0 written first). The vector $(0,0,1,1)/\sqrt{2}$ is what one gets with the qubit order reversed — conventions matter when reading state vectors.
Why the other options are incorrect.
(A) $(0,0,1,1)$ reverses the qubit order (qubit 1 written first).
(B) $(1,0,1,0)$ treats $R_Y(\pi)$ as the identity; it maps $|0\rangle$ to $|1\rangle$.
(D) $(0,1,0,-1)$ puts a minus sign on $|11\rangle$; $R_Y(\pi/2)\,|0\rangle$ has two positive amplitudes.
Key takeaway. Build each qubit's state, then take the Kronecker product in the stated qubit order.

Q.29 Single-correct MCQ · Unitarity

Which of the following matrices is *not* unitary?

(A) $\begin{pmatrix} c & -is \\ -is & c \end{pmatrix}$ with $c = \cos\frac{\pi}{8}$, $s = \sin\frac{\pi}{8}$ **(B)** ✓ $\frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$

(C) $\frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ **(D)** $\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$

Answer: (B) Detailed solution

Correct answer. **(B)** $\frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$

Concepts required. Unitarity: $U^\dagger U = I$, equivalently orthonormal columns; unitary matrices are invertible.

Relevant formulae. $U^\dagger U = I$; $|\det U| = 1$; H, S and $R_X(\theta) = \cos\frac{\theta}{2} I - i\sin\frac{\theta}{2} X$ are unitary.

Step-by-step explanation. U is unitary iff $U^\dagger U = I$, i.e. its columns are orthonormal. For $\frac{1}{\sqrt{2}}\left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ both columns are equal, $\frac{1}{\sqrt{2}}(1,1)$: they are not orthogonal, and $U^\dagger U = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right) \neq I$ (the matrix is singular, $\det = 0$). The others are the Hadamard H, the phase gate S and $R_X(\pi/4)$, all unitary. Only unitary gates (plus measurement) are allowed in a quantum circuit.

Why the other options are incorrect.

(A) This is $R_X(\pi/4)$: $\cos^2\frac{\pi}{8} + \sin^2\frac{\pi}{8} = 1$ and the columns are orthogonal.

(C) $H = \frac{1}{\sqrt{2}}\left(\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right)$: orthonormal columns, $H^2 = I$.

(D) $S = \mathrm{diag}(1, i)$: a diagonal matrix with entries of modulus 1 is unitary.

Key takeaway. Equal (or non-orthogonal) columns ⇒ not unitary ⇒ not a gate.

Q.30 Single-correct MCQ · Global vs. relative phase

Which pair of states gives *identical* outcome statistics for every possible measurement?

(A) ✓ $\frac{1}{\sqrt{2}}(|0\rangle + i\,|1\rangle)$ and $\frac{e^{i\pi/3}}{\sqrt{2}}(|0\rangle + i\,|1\rangle)$

(B) $\frac{1}{\sqrt{2}}(|0\rangle + i\,|1\rangle)$ and $\frac{1}{\sqrt{2}}(|0\rangle - i\,|1\rangle)$

(C) $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $\frac{1}{\sqrt{2}}(|0\rangle + i\,|1\rangle)$

(D) $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$

Answer: (A) Detailed solution

Correct answer. **(A)** $\frac{1}{\sqrt{2}}(|0\rangle + i\,|1\rangle)$ and $\frac{e^{i\pi/3}}{\sqrt{2}}(|0\rangle + i\,|1\rangle)$

Concepts required. Global phase is unobservable; relative phase is observable (interference, change of basis).

Relevant formulae. $|\langle m|e^{i\phi}\psi\rangle|^2 = |\langle m|\psi\rangle|^2$ for every $|m\rangle$.

Step-by-step explanation. Two state vectors describe the same physical state iff they differ by a *global* phase $e^{i\phi}$: every probability $|\langle m|\psi\rangle|^2$ is unchanged by $|\psi\rangle \to e^{i\phi}|\psi\rangle$. The first pair differs by $e^{i\pi/3}$. The other pairs differ by a *relative* phase between $|0\rangle$ and $|1\rangle$: they have different Bloch vectors ($\pm\hat{x}$; $\pm\hat{y}$; $\hat{x}$ vs. $\hat{y}$) and are distinguished by measuring X or Y (e.g. H then Z).

Why the other options are incorrect.

(B) $\frac{1}{\sqrt{2}}(|0\rangle \pm i\,|1\rangle)$ are the ±1 eigenstates of Y: a Y measurement distinguishes them perfectly.

(C) $|+\rangle$ and $\frac{1}{\sqrt{2}}(|0\rangle + i\,|1\rangle)$ have Bloch vectors $\hat{x}$ and $\hat{y}$; an X measurement gives $+1$ with probabilities 1 and $\frac{1}{2}$.

(D) $|+\rangle$ and $|-\rangle$ differ by a relative sign: an X measurement (or H then Z) distinguishes them perfectly.

Key takeaway. Only an overall factor $e^{i\phi}$ leaves all statistics unchanged.

Q.31 Single-correct MCQ · Conjugation $R_Y^\dagger Z R_Y$

For $R_Y(\theta) = e^{-i\theta Y/2}$, the conjugated observable $R_Y(\theta)^\dagger\, Z\, R_Y(\theta)$ equals

(A) ✓ $\cos\theta\, Z - \sin\theta\, X$ **(B)** $\cos\theta\, Z + \sin\theta\, X$ **(C)** $\cos\theta\, Z - \sin\theta\, Y$ **(D)** Z

Answer: (A) Detailed solution

Correct answer. **(A)** $\cos\theta\, Z - \sin\theta\, X$

Concepts required. Heisenberg picture: conjugating an observable by a rotation rotates it.

Relevant formulae. $R_Y(\theta) = \cos\frac{\theta}{2} I - i \sin\frac{\theta}{2} Y$; $YZ - ZY = 2iX$; $YZY = -Z$.

Step-by-step explanation. Use $R_Y(\theta) = \cos\frac{\theta}{2} I - i\sin\frac{\theta}{2} Y$ and $YZ = iX$, $ZY = -iX$: $R_Y^\dagger Z R_Y = (c + isY) Z (c - isY) = c^2 Z + ics(YZ - ZY) + s^2 YZY$ with $c = \cos\frac{\theta}{2}$, $s = \sin\frac{\theta}{2}$. $YZ - ZY = 2iX$, so $ics \cdot 2iX = -2csX = -\sin\theta X$; and $YZY = -Z$, so $s^2 YZY = -s^2 Z$. Total: $(c^2 - s^2)Z - \sin\theta X = \cos\theta Z - \sin\theta X$. *Check* on $|0\rangle$: $R_Y(\theta)|0\rangle$ has $\langle Z\rangle = \cos\theta$, and $\langle 0|(\cos\theta Z - \sin\theta X)|0\rangle = \cos\theta$. On $|+\rangle$: $R_Y(\theta)|+\rangle$ has $\langle Z\rangle = -\sin\theta$, matching $-\sin\theta\,\langle +|X|+\rangle$.

Why the other options are incorrect.

(B) $+\sin\theta X$ has the wrong sign; check on $|+\rangle$: $R_Y(\theta)|+\rangle$ has $\langle Z\rangle = -\sin\theta$.

(C) A rotation about Y mixes Z with X, not with Y (which is the rotation axis and unchanged).

(D) Z would mean R_Y commutes with Z, which is false (they anticommute generators).

Key takeaway. $R_Y^\dagger Z R_Y = \cos\theta Z - \sin\theta X$: the basis of the Heisenberg-picture analysis of variational circuits.

Q.32 Single-correct MCQ · Controlled-R_Z matrix

In the basis order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ (qubit 0 first), the controlled-$R_Z(\theta)$ gate with control qubit 0 and target qubit 1 — `rz.ctrl(theta, q[0], q[1])` in CUDA-Q — is

(A) $\mathrm{diag}\,(e^{-i\theta/2},\ e^{i\theta/2},\ 1,\ 1)$ **(B)** $\mathrm{diag}\,(1,\ 1,\ 1,\ e^{i\theta})$

(C) $\mathrm{diag}\,(1,\ e^{-i\theta/2},\ 1,\ e^{i\theta/2})$ **(D)** ✓ $\mathrm{diag}\,(1,\ 1,\ e^{-i\theta/2},\ e^{i\theta/2})$

Answer: (D) Detailed solution

Correct answer. **(D)** $\mathrm{diag}\,(1,\ 1,\ e^{-i\theta/2},\ e^{i\theta/2})$

Concepts required. Controlled gates act with U on the target only in the control-$|1\rangle$ subspace.

Relevant formulae. $C\text{-}U = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U$; $R_Z(\theta) = \mathrm{diag}(e^{-i\theta/2}, e^{i\theta/2})$.

Step-by-step explanation. Controlled-$U = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U$ with $U = R_Z(\theta) = \mathrm{diag}(e^{-i\theta/2}, e^{i\theta/2})$. For control 0 ($|00\rangle, |01\rangle$) nothing happens; for control 1 the target gets $e^{-i\theta/2}$ ($|10\rangle$) or $e^{i\theta/2}$ ($|11\rangle$). So the matrix is $\mathrm{diag}(1, 1, e^{-i\theta/2}, e^{i\theta/2})$. Note that it is *not* the controlled-phase $\mathrm{diag}(1, 1, 1, e^{i\theta})$: the two differ by the phase $e^{-i\theta/2}$ on the control-1 subspace, which is a *relative* phase and is observable. This gate is the first half of the QCNN pooling block.

Why the other options are incorrect.

(A) $\mathrm{diag}(e^{-i\theta/2}, e^{i\theta/2}, 1, 1)$ applies R_Z when the control is $|0\rangle$ (an anti-controlled gate).

(B) $\mathrm{diag}(1, 1, 1, e^{i\theta})$ is the controlled-*phase* gate; it differs from controlled-R_Z by the relative phase $e^{-i\theta/2}$ on the control-1 block.

(C) $\mathrm{diag}(1, e^{-i\theta/2}, 1, e^{i\theta/2})$ uses qubit 1 as control and qubit 0 as target.

Key takeaway. Controlled-$R_Z \neq$ controlled-phase: the difference is a relative phase that matters inside circuits.

Q.33 SINGLE-CORRECT MCQ · State-vector memory

A state-vector simulator stores every amplitude as a double-precision complex number (16 bytes). The memory needed for one 30-qubit state is

(A) ✓ 16 GiB (about 17.2 GB) **(B)** 1 GiB

(C) 480 bytes **(D)** 8 GiB

Answer: (A) Detailed solution

Correct answer. **(A)** 16 GiB (about 17.2 GB)

Concepts required. State-vector simulation stores all 2^n complex amplitudes.

Relevant formulae. Memory $= b \cdot 2^n$ bytes, $b = 16$ (complex128) or 8 (complex64); 1 GiB $= 2^{30}$ bytes.

Step-by-step explanation. An n-qubit state has 2^n amplitudes: $2^{30} = 1\,073\,741\,824$. Memory $= 16 \times 2^{30}$ bytes $= 2^{34}$ bytes $= 16$ GiB $\approx 17.2 \times 10^9$ bytes. Each additional qubit doubles it; single precision (8 bytes) halves it. The 8-qubit default of `QML.ipynb` needs only $16 \times 2^8 = 4096$ bytes.

Why the other options are incorrect.

(B) 1 GiB counts one byte per amplitude.

(C) 480 bytes $= 16 \times 30$ assumes memory linear in n; it is exponential.

(D) 8 GiB is single precision (8 bytes per amplitude).

Key takeaway. $16 \cdot 2^n$ bytes: 30 qubits = 16 GiB, each extra qubit doubles it.

Q.34 SINGLE-CORRECT MCQ · Two-parameter single-qubit circuit

A single qubit is prepared as $R_X(\theta_2)R_Y(\theta_1)\,|0\rangle$ (first R_Y, then R_X). The expectation value $f(\theta_1, \theta_2) = \langle Z \rangle$ is

(A) ✓ $\cos\theta_1 \cos\theta_2$ **(B)** $\cos\theta_1 - \sin\theta_1 \sin\theta_2$ **(C)** $\cos(\theta_1 + \theta_2)$ **(D)** $\frac{1}{2}(\cos\theta_1 + \cos\theta_2)$

Answer: (A) Detailed solution

Correct answer. **(A)** $\cos\theta_1 \cos\theta_2$

Concepts required. Single-qubit rotations act on the Bloch vector; rotations about different axes do not simply add.

Relevant formulae. $R_Y(\theta)$: $(x, z) \to (x\cos\theta + z\sin\theta,\ -x\sin\theta + z\cos\theta)$; $R_X(\theta)$: $(y, z) \to (y\cos\theta - z\sin\theta,\ y\sin\theta + z\cos\theta)$.

Step-by-step explanation. Follow the Bloch vector $\vec{r} = (\langle X\rangle, \langle Y\rangle, \langle Z\rangle)$. Start: $(0, 0, 1)$. $R_Y(\theta_1)$ rotates about y: $\vec{r} = (\sin\theta_1, 0, \cos\theta_1)$. $R_X(\theta_2)$ rotates about x: $(y, z) \to (y\cos\theta_2 - z\sin\theta_2,\ y\sin\theta_2 + z\cos\theta_2)$, so $z' = 0 \cdot \sin\theta_2 + \cos\theta_1 \cos\theta_2$. Hence $f = \cos\theta_1 \cos\theta_2$: rotations about *different* axes do not add angles. This is the simplest "variational" model: f is trigonometric in each parameter separately.

Why the other options are incorrect.

(B) $\cos\theta_1 - \sin\theta_1 \sin\theta_2$ mixes in the x component, which an R_X rotation does not move into z.

(C) $\cos(\theta_1 + \theta_2)$ would hold for two rotations about the *same* axis.

(D) The average of the two cosines has no basis; the second rotation acts on the already rotated vector.

Key takeaway. Track the Bloch vector gate by gate: $f = \cos\theta_1 \cos\theta_2$.

Q.35 MATCHING LISTS · Gate identities

Match each gate (List-I) with a correct property (List-II). $S = \mathrm{diag}(1, i)$.

List-I		List-II	
(P)	H	**(1)**	maps $\lvert 0\rangle \mapsto \lvert 1\rangle$ and $\lvert 1\rangle \mapsto -\lvert 0\rangle$
(Q)	S	**(2)**	equals $-iX$
(R)	$R_Y(\pi)$	**(3)**	conjugation exchanges X and Z: $(\cdot)Z(\cdot)^\dagger = X$
(S)	$R_X(\pi)$	**(4)**	conjugation maps X to Y: $(\cdot)X(\cdot)^\dagger = Y$

(A) P→2, Q→4, R→1, S→3
(B) P→3, Q→4, R→2, S→1
(C) ✓ P→3, Q→4, R→1, S→2
(D) P→4, Q→3, R→1, S→2

Answer: (C) **Detailed solution**

Correct answer. **(C)** P→3, Q→4, R→1, S→2
Concepts required. Gate identities: Clifford conjugation rules and rotation gates at angle π.
Relevant formulae. $HZH = X$; $SXS^\dagger = Y$; $R_P(\pi) = -iP$ for $P \in \{X, Y, Z\}$.
Step-by-step explanation.

- H: $HZH = X$ and $HXH = Z$ (3).
- S: $SXS^\dagger = \left(\begin{smallmatrix}1&0\\0&i\end{smallmatrix}\right)\left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right)\left(\begin{smallmatrix}1&0\\0&-i\end{smallmatrix}\right) = \left(\begin{smallmatrix}0&-i\\i&0\end{smallmatrix}\right) = Y$ (4).
- $R_Y(\pi) = \cos\frac{\pi}{2} I - i \sin\frac{\pi}{2} Y = -iY = \left(\begin{smallmatrix}0&-1\\1&0\end{smallmatrix}\right)$: $\lvert 0\rangle \mapsto \lvert 1\rangle$, $\lvert 1\rangle \mapsto -\lvert 0\rangle$ (1).
- $R_X(\pi) = \cos\frac{\pi}{2} I - i \sin\frac{\pi}{2} X = -iX$ (2): an X gate up to a global phase.

Hence P→3, Q→4, R→1, S→2.
Why the other options are incorrect.
(A) P→2 claims $H = -iX$; H is a different (Hermitian, real) matrix, and $R_X(\pi)$ is the gate equal to $-iX$.
(B) R→2, S→1 swaps the two rotations: $R_Y(\pi) = -iY$ is real ($\lvert 0\rangle \mapsto \lvert 1\rangle$, $\lvert 1\rangle \mapsto -\lvert 0\rangle$), while $R_X(\pi) = -iX$.
(D) P→4, Q→3 swaps H and S: H exchanges $X \leftrightarrow Z$, S maps $X \to Y$.
Key takeaway. H: $X \leftrightarrow Z$; S: $X \to Y$; $R_P(\pi) = -iP$.

Q.36 MATCHING LISTS · Bloch vectors

Match each state (List-I) with its Bloch vector $(\langle X\rangle, \langle Y\rangle, \langle Z\rangle)$ (List-II).

List-I		List-II	
(P)	$\lvert+\rangle = \frac{1}{\sqrt{2}}(\lvert 0\rangle + \lvert 1\rangle)$	**(1)**	$(0,\ 0,\ -1)$
(Q)	$\frac{1}{\sqrt{2}}(\lvert 0\rangle + i\lvert 1\rangle)$	**(2)**	$(\frac{1}{\sqrt{2}},\ 0,\ \frac{1}{\sqrt{2}})$
(R)	$\cos\frac{\pi}{8}\lvert 0\rangle + \sin\frac{\pi}{8}\lvert 1\rangle$	**(3)**	$(1,\ 0,\ 0)$
(S)	$\lvert 1\rangle$	**(4)**	$(0,\ 1,\ 0)$

(A) P→4, Q→3, R→2, S→1
(B) P→3, Q→4, R→1, S→2
(C) P→2, Q→4, R→3, S→1
(D) ✓ P→3, Q→4, R→2, S→1

Answer: (D) Detailed solution

Correct answer. **(D)** P→3, Q→4, R→2, S→1
Concepts required. Bloch-sphere coordinates of single-qubit pure states.
Relevant formulae. $\cos\frac{\vartheta}{2}|0\rangle + e^{i\varphi}\sin\frac{\vartheta}{2}|1\rangle \mapsto (\sin\vartheta\cos\varphi, \sin\vartheta\sin\varphi, \cos\vartheta)$.
Step-by-step explanation. A state $\cos\frac{\vartheta}{2}|0\rangle + e^{i\varphi}\sin\frac{\vartheta}{2}|1\rangle$ has Bloch vector $(\sin\vartheta\cos\varphi,\ \sin\vartheta\sin\varphi,\ \cos\vartheta)$.

- $|+\rangle$: $\vartheta=\frac{\pi}{2}$, $\varphi=0$: $(1,0,0)$ (3).
- $\frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)$: $\vartheta=\frac{\pi}{2}$, $\varphi=\frac{\pi}{2}$: $(0,1,0)$ (4).
- $\cos\frac{\pi}{8}|0\rangle + \sin\frac{\pi}{8}|1\rangle$: $\vartheta=\frac{\pi}{4}$, $\varphi=0$: $(\frac{1}{\sqrt{2}},0,\frac{1}{\sqrt{2}})$ (2) — this is the notebook's angle encoding of a pixel value $x=\frac{1}{4}$: $R_Y(\pi x)|0\rangle$.
- $|1\rangle$: south pole $(0,0,-1)$ (1).

Hence P→3, Q→4, R→2, S→1.
Why the other options are incorrect.
(A) P→4, Q→3 swaps $|+\rangle$ and $|+i\rangle$: a real positive relative amplitude points along $+x$, a factor i along $+y$.
(B) R→1, S→2 puts state R at the south pole and $|1\rangle$ at polar angle $\frac{\pi}{4}$; $|1\rangle$ is the south pole, and state R has $\vartheta=\frac{\pi}{4}$.
(C) P→2, R→3 confuses $\vartheta=\frac{\pi}{4}$ (state R) with $\vartheta=\frac{\pi}{2}$ ($|+\rangle$): the half-angle $\frac{\pi}{8}$ means a polar angle $\frac{\pi}{4}$.
Key takeaway. The amplitude angle is *half* the Bloch polar angle; the relative phase is the azimuth.

Q.37 ASSERTION–REASON · Same statistics, different states

Assertion (A): For the state $R_X(\theta)|0\rangle$ one has $\langle Z\rangle = \cos\theta$, so θ and $-\theta$ give identical statistics for a measurement in the computational basis.

Reason (R): $R_X(\theta)|0\rangle$ and $R_X(-\theta)|0\rangle$ are the same quantum state.

(A) Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A
(B) Both A and R are true, but R is *not* the correct explanation of A
(C) ✓ A is true, but R is false
(D) A is false, but R is true

Answer: (C) Detailed solution

Correct answer. **(C)** A is true, but R is false
Concepts required. Equal Z-statistics do not imply equal states; relative phases are invisible to Z measurements.
Relevant formulae. $R_X(\theta)|0\rangle = \cos\frac{\theta}{2}|0\rangle - i\sin\frac{\theta}{2}|1\rangle$; Bloch vector $(0,-\sin\theta,\cos\theta)$.
Step-by-step explanation. $R_X(\theta)|0\rangle = \cos\frac{\theta}{2}|0\rangle - i\sin\frac{\theta}{2}|1\rangle$, so $P(0)=\cos^2\frac{\theta}{2}$, $P(1)=\sin^2\frac{\theta}{2}$ and $\langle Z\rangle=\cos\theta$, an even function: **A is true**. $R_X(-\theta)|0\rangle = \cos\frac{\theta}{2}|0\rangle + i\sin\frac{\theta}{2}|1\rangle$ differs by the *relative* phase -1 on $|1\rangle$; the Bloch vectors are $(0,\mp\sin\theta,\cos\theta)$, distinguishable by measuring Y. **R is false** (for $\sin\theta\neq 0$). Answer: (C). (Same Z-statistics, different states: a Z readout alone cannot tell them apart.)
Why the other options are incorrect.
(A) needs R true, but the two states differ in $\langle Y\rangle$.
(B) needs R true as well; it is false.
(D) needs A false, but $\cos\theta$ is even, so the Z-statistics are indeed equal.
Key takeaway. Same measurement statistics in one basis ≠ same state.

Q.38 Statement I / II · CNOT and the product-state criterion

Statement I: A CNOT gate acting on a product state always produces an entangled state.

Statement II: A pure two-qubit state $c_{00}|00\rangle + c_{01}|01\rangle + c_{10}|10\rangle + c_{11}|11\rangle$ is a product state if and only if $c_{00}c_{11} - c_{01}c_{10} = 0$.

(A) Both Statement I and Statement II are true

(B) Both Statement I and Statement II are false

(C) Statement I is true, but Statement II is false

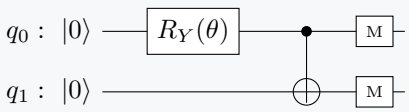
(D) ✓ Statement I is false, but Statement II is true

Answer: (D) Detailed solution

Correct answer. **(D)** Statement I is false, but Statement II is true
Concepts required. Entangling power of CNOT and the determinant criterion for product states.
Relevant formulae. $\text{CNOT}|a\rangle|b\rangle = |a\rangle|a \oplus b\rangle$; product $\Leftrightarrow c_{00}c_{11} = c_{01}c_{10}$.
Step-by-step explanation. **I is false:** $\text{CNOT}|00\rangle = |00\rangle$, $\text{CNOT}|1\rangle|+\rangle = |1\rangle|+\rangle$ and $\text{CNOT}|+\rangle|+\rangle = |+\rangle|+\rangle$ are all product states. CNOT entangles only for suitable inputs, e.g. $|+\rangle|0\rangle \mapsto |\Phi^+\rangle$. **II is true:** $(a_0|0\rangle + a_1|1\rangle) \otimes (b_0|0\rangle + b_1|1\rangle)$ has $c_{ab} = a_a b_b$, a rank-1 matrix with determinant 0; conversely a 2×2 matrix with determinant 0 has rank ≤ 1 and factorises as an outer product $\mathbf{a}\mathbf{b}^{\mathsf{T}}$. Answer: (D).
Why the other options are incorrect.
(A) needs I true, but $\text{CNOT}|00\rangle = |00\rangle$ is a product state.
(B) needs II false, but the rank-1 (determinant) criterion is exact for pure two-qubit states.
(C) needs I true and II false: both are reversed.
Key takeaway. CNOT entangles only superposed controls with suitable targets; $\det C = 0$ tests product states.

Paragraph for Questions 39 and 40

A two-qubit parameterized circuit starts in $|00\rangle$, applies $R_Y(\theta)$ to qubit 0 and then a CNOT with control qubit 0 and target qubit 1:



Both qubits are then measured in the computational basis; $0 \leq \theta \leq \pi$.

Q.39 Paragraph based · Correlations of R_Y + CNOT state

The expectation values $\langle Z_1\rangle$ and $\langle Z_0 Z_1\rangle$ of the final state are

(A) ✓ $\cos\theta$ and 1 **(B)** 1 and $\cos\theta$ **(C)** $\cos^2(\theta/2)$ and 1 **(D)** $\cos\theta$ and $\cos^2\theta$

Answer: (A) Detailed solution

Correct answer. **(A)** $\cos\theta$ and 1
Concepts required. Building an entangled state with R_Y + CNOT and computing correlations.
Relevant formulae. $|\psi\rangle = \cos\frac{\theta}{2}|00\rangle + \sin\frac{\theta}{2}|11\rangle$; $\langle Z_1\rangle = \sum_{ab}|c_{ab}|^2(-1)^b$, $\langle Z_0 Z_1\rangle = \sum|c_{ab}|^2(-1)^{a+b}$.
Step-by-step explanation. $R_Y(\theta)|0\rangle \otimes |0\rangle = \cos\frac{\theta}{2}|00\rangle + \sin\frac{\theta}{2}|10\rangle$; the CNOT flips qubit 1 when qubit 0 is 1: $|\psi\rangle = \cos\frac{\theta}{2}|00\rangle + \sin\frac{\theta}{2}|11\rangle$. $\langle Z_1\rangle = \cos^2\frac{\theta}{2} - \sin^2\frac{\theta}{2} = \cos\theta$. Both components have even parity, so $\langle Z_0 Z_1\rangle = 1$ for every θ: the outcomes of the two qubits always agree. (Product of the

marginals would give $\cos^2\theta$, which is wrong because the qubits are correlated.)

Why the other options are incorrect.

(B) $\langle Z_1\rangle = 1$ would mean qubit 1 is always 0; it is 1 with probability $\sin^2\frac{\theta}{2}$.

(C) $\cos^2(\theta/2)$ is $P(q_1 = 0)$, not $\langle Z_1\rangle = P(0) - P(1)$.

(D) $\cos^2\theta$ multiplies the marginals; the outcomes are perfectly correlated, so the parity is always even.

Key takeaway. $\langle Z_1\rangle = \cos\theta$, $\langle Z_0 Z_1\rangle = 1$: perfectly correlated outcomes.

Q.40 PARAGRAPH BASED · Purity and maximal entanglement

The value of θ in $(0, \pi]$ at which the two qubits are *maximally* entangled, and the purity $\operatorname{tr}\rho_1^2$ of the reduced state of qubit 1 at that value, are

(A) $\theta = \pi/2$ and $\operatorname{tr}\rho_1^2 = 1$

(B) ✓ $\theta = \pi/2$ and $\operatorname{tr}\rho_1^2 = \frac{1}{2}$

(C) $\theta = \pi$ and $\operatorname{tr}\rho_1^2 = \frac{1}{2}$

(D) $\theta = \pi/4$ and $\operatorname{tr}\rho_1^2 = \frac{3}{4}$

Answer: (B) Detailed solution

Correct answer. (B) $\theta = \pi/2$ and $\operatorname{tr}\rho_1^2 = \frac{1}{2}$

Concepts required. Entanglement of a pure bipartite state measured by the mixedness (purity) of the reduced state.

Relevant formulae. $\rho_1 = \operatorname{diag}(\cos^2\frac{\theta}{2}, \sin^2\frac{\theta}{2})$; $\operatorname{tr}\rho_1^2 = \frac{1+\cos^2\theta}{2} \in [\frac{1}{2}, 1]$.

Step-by-step explanation. Tracing out qubit 0 from $\cos\frac{\theta}{2}|00\rangle + \sin\frac{\theta}{2}|11\rangle$ gives $\rho_1 = \operatorname{diag}(\cos^2\frac{\theta}{2}, \sin^2\frac{\theta}{2})$ and purity $\operatorname{tr}\rho_1^2 = \cos^4\frac{\theta}{2} + \sin^4\frac{\theta}{2} = \frac{1+\cos^2\theta}{2}$. It is minimal (most mixed, most entangled) when $\cos\theta = 0$: $\theta = \frac{\pi}{2}$, giving $\frac{1}{2}$ — the Bell state $|\Phi^+\rangle$. At $\theta = \pi$ the state is $|11\rangle$ (product, purity 1); at $\theta = \frac{\pi}{4}$ the purity is $\frac{1+1/2}{2} = \frac{3}{4}$.

Why the other options are incorrect.

(A) Purity 1 means a pure reduced state, i.e. *no* entanglement.

(C) $\theta = \pi$ gives $|11\rangle$, a product state with purity 1.

(D) At $\theta = \frac{\pi}{4}$ the state is only partially entangled (purity $\frac{3}{4}$).

Key takeaway. Maximal entanglement ⇔ maximally mixed reduced state ($\operatorname{tr}\rho^2 = \frac{1}{2}$ for a qubit).

SECTION C · Quantum Machine Learning Fundamentals Q.41–60

Q.41 Single-correct MCQ · Angle encoding of a pixel

`QML.ipynb` encodes a pixel value $x \in [0,1]$ with `ry(np.pi * x, q)` on a qubit in $|0\rangle$. For a pixel of brightness $x = \frac{1}{3}$ the probability of measuring 1 on that qubit and its $\langle Z \rangle$ (before any other gate) are

(A) ✓ $P(1) = \frac{1}{4}$, $\langle Z \rangle = \frac{1}{2}$
(B) $P(1) = \frac{1}{2}$, $\langle Z \rangle = 0$
(C) $P(1) = \frac{1}{3}$, $\langle Z \rangle = \frac{1}{3}$
(D) $P(1) = \frac{3}{4}$, $\langle Z \rangle = -\frac{1}{2}$

Answer: (A) Detailed solution

Correct answer. **(A)** $P(1) = \frac{1}{4}$, $\langle Z \rangle = \frac{1}{2}$
Concepts required. Angle encoding writes a feature into a rotation angle; the half-angle appears in the amplitudes.
Relevant formulae. $R_Y(\pi x)\,|0\rangle = \cos\frac{\pi x}{2}\,|0\rangle + \sin\frac{\pi x}{2}\,|1\rangle$; $P(1) = \sin^2\frac{\pi x}{2}$, $\langle Z \rangle = \cos \pi x$.
Step-by-step explanation. $R_Y(\pi x)\,|0\rangle = \cos\frac{\pi x}{2}\,|0\rangle + \sin\frac{\pi x}{2}\,|1\rangle$. For $x = \frac{1}{3}$ the half-angle is $\frac{\pi}{6}$: $P(1) = \sin^2\frac{\pi}{6} = \frac{1}{4}$, $P(0) = \frac{3}{4}$, so $\langle Z \rangle = \frac{3}{4} - \frac{1}{4} = \frac{1}{2} = \cos\frac{\pi}{3} = \cos(\pi x)$. A dark pixel ($x = 0$) stays $|0\rangle$, a lit pixel ($x = 1$) becomes $|1\rangle$, intermediate values give superpositions.
Why the other options are incorrect.
(B) $P(1) = \frac{1}{2}$ would correspond to $x = \frac{1}{2}$.
(C) Angle encoding does not make $P(1)$ equal to the pixel value; $P(1) = \sin^2\frac{\pi x}{2}$ is non-linear in x.
(D) $P(1) = \frac{3}{4}$ uses $\sin^2(\pi x)$, forgetting the half-angle of R_Y.
Key takeaway. Notebook encoding: $\langle Z_i \rangle = \cos(\pi x_i)$ before the QCNN layers.

Q.42 Single-correct MCQ · Amplitude encoding

The vector $\mathbf{x} = (1, 2, 4, 2)$ is amplitude-encoded: $|\psi_{\mathbf{x}}\rangle = \sum_j \frac{x_j}{\|\mathbf{x}\|}\,|j\rangle$ with $|0\rangle = |00\rangle, |1\rangle = |01\rangle, |2\rangle = |10\rangle, |3\rangle = |11\rangle$ (qubit 0 written first). The number of qubits and the probability that a measurement of qubit 0 gives 1 are

(A) 2 qubits; 0.32 **(B)** 2 qubits; 0.60 **(C)** 4 qubits; 0.80 **(D)** ✓ 2 qubits; 0.80

Answer: (D) Detailed solution

Correct answer. **(D)** 2 qubits; 0.80
Concepts required. Amplitude encoding stores a normalised vector in the amplitudes of $\log_2 d$ qubits.
Relevant formulae. $|\psi_{\mathbf{x}}\rangle = \frac{1}{\|\mathbf{x}\|}\sum_j x_j\,|j\rangle$; $P(q_0 = 1) = \sum_{j:\,q_0(j)=1} |x_j|^2 / \|\mathbf{x}\|^2$.
Step-by-step explanation. Four amplitudes need $\log_2 4 = 2$ qubits. $\|\mathbf{x}\| = \sqrt{1+4+16+4} = 5$, so the amplitudes are $(0.2, 0.4, 0.8, 0.4)$ on $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. Qubit 0 is 1 in $|10\rangle$ and $|11\rangle$: $P = 0.8^2 + 0.4^2 = 0.64 + 0.16 = 0.80$. (Qubit 1 is 1 in $|01\rangle, |11\rangle$: $0.16 + 0.16 = 0.32$.) Amplitude encoding is exponentially compact in qubits, but preparing a general such state needs $O(2^n)$ gates.
Why the other options are incorrect.
(A) 0.32 is the probability for qubit 1 (basis states $|01\rangle, |11\rangle$), not qubit 0.
(B) 0.60 averages the amplitudes 0.8 and 0.4 instead of adding their squares.
(C) Four qubits would be angle/basis encoding; amplitude encoding needs only $\log_2 4 = 2$.
Key takeaway. Normalise, then sum |amplitude|^2 over the basis states with the wanted bit.

Q.43 Single-correct MCQ · Qubit cost of encodings

A data point has $d = 256$ real features, each stored with 8-bit precision. The numbers of qubits needed by *basis* encoding (every bit on its own qubit), *angle* encoding (one feature per qubit) and *amplitude* encoding are

(A) 256, 256, 8 **(B)** 8, 256, 2048 **(C)** ✓ 2048, 256, 8 **(D)** 2048, 8, 256

Answer: (C) Detailed solution

Correct answer. **(C)** 2048, 256, 8
Concepts required. Resource comparison of basis, angle and amplitude encoding.
Relevant formulae. Qubits: basis $= d \cdot b$ (b bits per feature), angle $= d$, amplitude $= \lceil \log_2 d \rceil$.
Step-by-step explanation. Basis encoding writes the bit string into computational basis states: $256 \times 8 = 2048$ bits, one qubit each. Angle encoding uses one rotation angle per feature: 256 qubits. Amplitude encoding stores $256 = 2^8$ numbers as the amplitudes of 8 qubits. Fewer qubits come at the price of a deeper, more complicated state-preparation circuit.
Why the other options are incorrect.
(A) 256 basis qubits ignores the 8 bits per feature.
(B) The order is reversed: basis encoding is the most qubit-hungry, amplitude the least.
(D) Angle and amplitude are swapped: angle encoding is linear in d, amplitude encoding logarithmic.
Key takeaway. Qubits: basis $\gg$ angle $\gg$ amplitude; circuit depth runs the other way.

Q.44 Single-correct MCQ · Fidelity kernel of angle encoding

With the notebook's angle encoding $|\psi(\mathbf{x})\rangle = \bigotimes_i R_Y(\pi x_i)\,|0\rangle$, the fidelity (quantum) kernel $k(\mathbf{x}, \mathbf{x}') = |\langle\psi(\mathbf{x})|\psi(\mathbf{x}')\rangle|^2$ for $\mathbf{x} = (1, 0.5)$ and $\mathbf{x}' = (0.5, 0)$ is

(A) $\frac{1}{16}$ **(B)** ✓ $\frac{1}{4}$ **(C)** $\frac{1}{2}$ **(D)** 0

Answer: (B) Detailed solution

Correct answer. **(B)** $\frac{1}{4}$
Concepts required. Fidelity kernel of a product (angle) encoding factorises over qubits.
Relevant formulae. $\langle\psi(x)|\psi(x')\rangle = \cos\frac{\pi(x-x')}{2}$ per qubit; $k = \prod_i \cos^2\frac{\pi(x_i - x_i')}{2}$.
Step-by-step explanation. For one qubit $\langle\psi(x)|\psi(x')\rangle = \cos\frac{\pi x}{2}\cos\frac{\pi x'}{2} + \sin\frac{\pi x}{2}\sin\frac{\pi x'}{2} = \cos\frac{\pi(x-x')}{2}$. For a product state the overlap is the product: $k = \prod_i \cos^2\frac{\pi(x_i - x_i')}{2}$. Here $x_0 - x_0' = 0.5$ and $x_1 - x_1' = 0.5$, so $k = \cos^2\frac{\pi}{4}\cdot\cos^2\frac{\pi}{4} = \frac{1}{2}\cdot\frac{1}{2} = \frac{1}{4}$. (The overlap itself is $\frac{1}{2}$; the kernel is its square.) Note that this kernel has a simple closed form — it can be evaluated classically in $O(d)$ time.
Why the other options are incorrect.
(A) $\frac{1}{16}$ squares the kernel again ($|\langle\psi|\psi'\rangle|^4$).
(C) $\frac{1}{2}$ is the overlap $|\langle\psi|\psi'\rangle|$, not its square.
(D) 0 would need orthogonal states, i.e. a feature difference of 1 on some qubit.
Key takeaway. Product encodings give product kernels — classically cheap, so no quantum advantage from them.

Q.45 SINGLE-CORRECT MCQ · Size of the kernel computation

A quantum-kernel SVM is trained on $M = 20$ points. Every kernel entry $k_{ij} = |\langle\psi_i|\psi_j\rangle|^2$, with $|\psi_i\rangle = |\psi(\mathbf{x}_i)\rangle$, is estimated by its own circuit. Using $k(\mathbf{x}, \mathbf{x}) = 1$ and the symmetry $k_{ij} = k_{ji}$, the number of distinct circuits needed for the training Gram matrix is

(A) ✓ 190 **(B)** 400 **(C)** 380 **(D)** 20

Answer: (A) Detailed solution

Correct answer. **(A)** 190

Concepts required. Cost of building a kernel (Gram) matrix with a quantum computer.

Relevant formulae. Distinct off-diagonal entries of a symmetric $M \times M$ matrix: $\binom{M}{2} = \frac{M(M-1)}{2}$.

Step-by-step explanation. The 20×20 Gram matrix has 400 entries. The 20 diagonal entries are known ($= 1$), leaving 380 off-diagonal entries, and symmetry halves them: $\binom{20}{2} = \frac{20 \cdot 19}{2} = 190$ circuits (each run with many shots). Prediction on a new point needs another $M = 20$ kernel values. The quadratic growth $O(M^2)$ is a practical limitation of kernel methods.

Why the other options are incorrect.

(B) $400 = M^2$ ignores the known diagonal and the symmetry.

(C) $380 = M^2 - M$ ignores the symmetry $k_{ij} = k_{ji}$.

(D) 20 counts the training points, not the pairs.

Key takeaway. Kernel training needs $O(M^2)$ circuit estimates; prediction $O(M)$ per new point.

Q.46 SINGLE-CORRECT MCQ · Swap test

In a swap test between the states $|\psi\rangle$ and $|\phi\rangle$ the ancilla is measured in state 0 with probability 0.8. The fidelity kernel $|\langle\psi|\phi\rangle|^2$ is

(A) 0.3 **(B)** 0.8 **(C)** 0.89 **(D)** ✓ 0.6

Answer: (D) Detailed solution

Correct answer. **(D)** 0.6

Concepts required. The swap test estimates the squared overlap of two states from one ancilla qubit.

Relevant formulae. $P(\text{ancilla} = 0) = \frac{1}{2}(1 + |\langle\psi|\phi\rangle|^2)$.

Step-by-step explanation. The swap test (Hadamard on the ancilla, controlled-SWAP, Hadamard, measure) gives $P(0) = \frac{1}{2}(1 + |\langle\psi|\phi\rangle|^2)$. Solving: $|\langle\psi|\phi\rangle|^2 = 2P(0) - 1 = 2(0.8) - 1 = 0.6$. Orthogonal states give $P(0) = \frac{1}{2}$, identical states $P(0) = 1$. (The compute–uncompute test $U(\mathbf{x}')^\dagger U(\mathbf{x})$ instead gives $|\langle\psi|\phi\rangle|^2$ directly as the probability of the all-zero outcome.)

Why the other options are incorrect.

(A) 0.3 uses $P(0) - \frac{1}{2}$ without the factor 2.

(B) 0.8 is $P(0)$ itself, not the overlap.

(C) $0.89 = \sqrt{0.8}$ has no meaning here.

Key takeaway. $|\langle\psi|\phi\rangle|^2 = 2P(0) - 1$.

Q.47 SINGLE-CORRECT MCQ · Two-term parameter-shift rule

A parameter θ enters a circuit only through one gate $R_Y(\theta)$. The measured expectation values are $f(\theta+\frac{\pi}{2}) = 0.30$ and $f(\theta-\frac{\pi}{2}) = -0.50$. The exact derivative $f'(\theta)$ is

(A) -0.40 **(B)** 0.25 **(C)** ✓ 0.40 **(D)** 0.80

Answer: (C) Detailed solution

Correct answer. **(C)** 0.40

Concepts required. The parameter-shift rule gives exact derivatives of expectation values for Pauli-rotation gates.

Relevant formulae. $f'(\theta) = \frac{1}{2}[f(\theta+\frac{\pi}{2}) - f(\theta-\frac{\pi}{2})]$ for a gate $e^{-i\theta P/2}$, $P^2 = I$.

Step-by-step explanation. The two-term parameter-shift rule for a gate $e^{-i\theta P/2}$ (P a Pauli operator) is $f'(\theta) = \frac{1}{2}[f(\theta+\frac{\pi}{2}) - f(\theta-\frac{\pi}{2})]$. Here $\frac{1}{2}(0.30-(-0.50)) = 0.40$. The rule is *exact* (no step-size error), because $f(\theta) = a + b\cos\theta + c\sin\theta$: $\frac{1}{2}[f(\theta+\frac{\pi}{2}) - f(\theta-\frac{\pi}{2})] = \frac{1}{2}[-2b\sin\theta + 2c\cos\theta] = f'(\theta)$.

Why the other options are incorrect.

(A) -0.40 subtracts in the wrong order.

(B) $0.25 \approx 0.8/\pi$ treats the shifts as a finite difference with step π, which is not how the rule works.

(D) 0.80 forgets the factor $\frac{1}{2}$.

Key takeaway. Two circuit runs, shifted by $\pm\frac{\pi}{2}$, give the exact derivative — no step-size error.

Q.48 SINGLE-CORRECT MCQ · Shared parameters in the QCNN

In the default 8-qubit QCNN of `QML.ipynb`, the parameter $\theta_{1,1}$ = `theta[0]` is used by how many R_Y gates, and how many circuit evaluations does an exact parameter-shift derivative with respect to it need for one image (shifting each occurrence separately)?

(A) 11 gates; 22 circuits **(B)** 4 gates; 8 circuits

(C) ✓ 7 gates; 14 circuits **(D)** 1 gate; 2 circuits

Answer: (C) Detailed solution

Correct answer. **(C)** 7 gates; 14 circuits

Concepts required. Weight sharing: one parameter in many gates; the derivative is the sum over occurrences (product rule).

Relevant formulae. $\partial_\theta f = \sum_g \partial_{\theta^{(g)}} f$; each R_Y occurrence: 2 shifted circuits.

Step-by-step explanation. `theta[0]` is the first parameter of layer 1; it is applied by `ry(theta[t], qubits[a])` in *every* convolution block of that layer. Layer 1 acts on all 8 qubits: pairs $(0,1),(2,3),(4,5),(6,7)$ and then $(1,2),(3,4),(5,6)$, i.e. $4+3=7$ blocks, so 7 gates share $\theta_{1,1}$ (weight sharing, as in a CNN filter). By the product rule $\partial_\theta f = \sum_{g=1}^{7} \partial_{\theta^{(g)}} f$, and each occurrence needs the two-term shift (2 circuits): $7 \times 2 = 14$ circuits. Shifting the shared value once (2 circuits) does *not* give the derivative.

Why the other options are incorrect.

(A) 11 counts the convolution blocks of *all* layers; layers 2 and 3 use `theta[4]` and `theta[8]` instead.

(B) 4 counts only the even pairs $(0,1),(2,3),(4,5),(6,7)$ and forgets the odd pairs $(1,2),(3,4),(5,6)$.

(D) One gate would hold without weight sharing; in the QCNN every convolution block of layer 1 uses `theta[0]`.

Key takeaway. Shared parameters need one shift pair per occurrence: $7 \times 2 = 14$ circuits for $\theta_{1,1}$.

Q.49 Single-correct MCQ · QCNN depth and parameters

Using the notebook's layout rule (pool pairs of active qubits, keep the sinks; an odd last qubit moves on unchanged; 4 parameters per layer), a QCNN on $N = 20$ qubits (e.g. 4×5 pixels) has

(A) 4 layers and 16 parameters
(B) 19 layers and 76 parameters
(C) 5 layers and 40 parameters
(D) ✓ 5 layers and 20 parameters

Answer: (D) Detailed solution

Correct answer. (D) 5 layers and 20 parameters
Concepts required. QCNN depth: pooling halves (rounding up) the number of active qubits until one remains.
Relevant formulae. $n \to \lceil n/2 \rceil$ per layer; layers $= \lceil \log_2 N \rceil$; parameters $= 4\times$layers.
Step-by-step explanation. Track the number of active qubits: pooling keeps $\lceil n/2 \rceil$ of n (the sinks, plus the odd last qubit). $20 \to 10 \to 5 \to 3 \to 2 \to 1$: five layers. With four shared parameters per layer: $4 \times 5 = 20$ parameters. In general the layer count is $\lceil \log_2 N \rceil$ and the parameter count $4\lceil \log_2 N \rceil$ — logarithmic in N, while a dense classical layer on N inputs has $O(N)$ weights.
Why the other options are incorrect.
(A) 4 layers would leave 2 qubits from 20 ($20 \to 10 \to 5 \to 3 \to 2$); one more pooling is needed.
(B) 19 layers would pool away one qubit per layer; the QCNN halves the register.
(C) 40 parameters counts 8 per layer; each layer has only 4 shared parameters.
Key takeaway. $20 \to 10 \to 5 \to 3 \to 2 \to 1$: 5 layers, 20 parameters.

Q.50 Single-correct MCQ · Pooling block (code)

The pooling block of `QML.ipynb` on source qubit a and sink qubit b is `rz.ctrl(t3, q[a], q[b]); x(q[a]); rx.ctrl(t4, q[a], q[b]); x(q[a])`. As an operator it equals

(A) $R_Z(t_3)_b\, R_X(t_4)_b$ (independent of qubit a)
(B) $|1\rangle\langle 1|_b \otimes R_Z(t_3)_a + |0\rangle\langle 0|_b \otimes R_X(t_4)_a$
(C) $|0\rangle\langle 0|_a \otimes R_Z(t_3)_b + |1\rangle\langle 1|_a \otimes R_X(t_4)_b$
(D) ✓ $|1\rangle\langle 1|_a \otimes R_Z(t_3)_b + |0\rangle\langle 0|_a \otimes R_X(t_4)_b$

Answer: (D) Detailed solution

Correct answer. (D) $|1\rangle\langle 1|_a \otimes R_Z(t_3)_b + |0\rangle\langle 0|_a \otimes R_X(t_4)_b$
Concepts required. Reading CUDA-Q code: `x`-conjugation turns a controlled gate into an anti-controlled one.
Relevant formulae. $(X \otimes I)\, C\text{-}U\, (X \otimes I) = |1\rangle\langle 1| \otimes I + |0\rangle\langle 0| \otimes U$.
Step-by-step explanation. `rz.ctrl(t3, q[a], q[b])` $= |0\rangle\langle 0|_a \otimes I + |1\rangle\langle 1|_a \otimes R_Z(t_3)$. The pair `x(q[a]) ... x(q[a])` turns the following controlled gate into an *anti*-controlled one (it acts when a is $|0\rangle$): $|1\rangle\langle 1|_a \otimes I + |0\rangle\langle 0|_a \otimes R_X(t_4)$. Multiplying the two (they act on orthogonal control subspaces): $|1\rangle\langle 1|_a \otimes R_Z(t_3) + |0\rangle\langle 0|_a \otimes R_X(t_4)$. Qubit a is never used again; by the deferred-measurement principle this equals "measure a, then rotate b according to the result".
Why the other options are incorrect.
(A) The rotations are controlled; on their own they would not depend on the source qubit at all.
(B) Source and sink are swapped: `q[a]` is the control, `q[b]` the target.
(C) This swaps the roles: `rz.ctrl` without the X sandwich acts when a is $|1\rangle$.
Key takeaway. Pooling = "if $a = 1$ rotate b about Z, else about X", then forget a.

Q.51 Single-correct MCQ · QCNN gate count

For $N = 16$ qubits the notebook's layout has 26 convolution blocks and 15 pooling blocks. Counting the N encoding rotations and every gate of the blocks as `cudaq.estimate_resources` does, the circuit has

(A) 138 gates **(B)** ✓ 154 gates **(C)** 124 gates **(D)** 170 gates

Answer: (B) Detailed solution

Correct answer. (B) 154 gates
Concepts required. Counting gates from the layout: encoding + blocks.
Relevant formulae. Gates $= N + 3n_{\text{conv}} + 4n_{\text{pool}}$ (conv: 2 `ry` + `cx`; pool: `crz`, 2 `x`, `crx`).
Step-by-step explanation. Per convolution block: 2 `ry` + 1 CNOT = 3 gates. Per pooling block: `crz`, `x`, `crx`, `x` = 4 gates. Total $= N + 3n_{\text{conv}} + 4n_{\text{pool}} = 16 + 3(26) + 4(15) = 16 + 78 + 60 = 154$. *Check* with the default: $8 + 3(11) + 4(7) = 8 + 33 + 28 = 69$, the recorded value. (Doubling 69 gives 138, but the block counts do not scale linearly.)
Why the other options are incorrect.
(A) 138 doubles the 8-qubit count; block counts grow as $\approx 2N$ and N, not exactly by a factor 2.
(C) 124 forgets the two X gates of every pooling block.
(D) 170 counts the 16 encoding rotations twice.
Key takeaway. $16 + 78 + 60 = 154$; the formula reproduces the recorded 69 gates at $N = 8$.

Q.52 Single-correct MCQ · MSE vs. accuracy

A classifier with outputs in $[-1, 1]$ and prediction sign f gives $f = 0.6,\ -0.2,\ 0.9,\ 0.1$ on four images with labels $y = +1,\ -1,\ +1,\ -1$. Its mean squared error (as in `QML.ipynb`) and accuracy are

(A) ✓ 0.505 and 75 % **(B)** 2.02 and 75 % **(C)** 0.60 and 75 % **(D)** 0.505 and 50 %

Answer: (A) Detailed solution

Correct answer. (A) 0.505 and 75 %
Concepts required. MSE on real-valued outputs vs. sign-based accuracy.
Relevant formulae. $\mathcal{L} = \frac{1}{M}\sum(f_m - y_m)^2$; accuracy $= \frac{1}{M}\#\{m : \text{sign}\, f_m = y_m\}$.
Step-by-step explanation. Errors $f - y$: $-0.4,\ 0.8,\ -0.1,\ 1.1$; squares 0.16, 0.64, 0.01, 1.21; sum 2.02; mean $\mathcal{L} = 0.505$. Signs: $+, -, +, +$ against labels $+, -, +, -$: three correct, accuracy 75 %. The second image is classified correctly even though its error is large (0.8): MSE punishes small margins, accuracy only looks at the sign.
Why the other options are incorrect.
(B) 2.02 is the sum of squared errors, not the mean.
(C) 0.60 is the mean *absolute* error.
(D) 50 % counts the second image as wrong; $\text{sign}(-0.2) = -1 = y$, correct.
Key takeaway. A correct sign with a small margin still costs MSE.

Q.53 Single-correct MCQ · Circuit count and run time of the notebook

With its defaults (20 training images, `MAX_ITERATIONS = 100` loss evaluations, 20 test images) and the recorded time of 1.7 ms per circuit on `qpp-cpu`, the number of circuits `QML.ipynb` runs for training plus testing and the resulting estimate of the run time are

(A) ✓ 2020 circuits, about 3.4 s
(B) 100 circuits, about 0.17 s
(C) 2020 circuits, about 34 s
(D) 2000 circuits, about 0.17 s

Answer: (A) Detailed solution

Correct answer. (A) 2020 circuits, about 3.4 s
Concepts required. Cost model of hybrid training: circuits per loss evaluation = number of training images.
Relevant formulae. Circuits = $M_{\text{train}} \times n_{\text{eval}} + M_{\text{test}}$; time ≈ circuits × time per circuit.
Step-by-step explanation. Every loss evaluation runs the QCNN once per training image: $100 \times 20 = 2000$ circuits; evaluating the test set adds 20: 2020 circuits (the notebook prints exactly this number). Time: 2020×1.7 ms $= 3.43$ s; the recorded training time was 3.4 s. The classical optimizer itself costs almost nothing — simulation dominates.
Why the other options are incorrect.
(B) 100 circuits counts loss evaluations, but each evaluation runs one circuit per image.
(C) 34 s is ten times too large (2020×1.7 ms $= 3.4$ s).
(D) 2000 circuits forgets the 20 test circuits, and 0.17 s is 100×1.7 ms.
Key takeaway. 2020×1.7 ms ≈ 3.4 s — the recorded training time.

Q.54 Single-correct MCQ · Data re-uploading frequencies

A single-qubit model alternates trainable unitaries with the data-encoding gate $R_Y(x)$, which appears $L = 3$ times ("data re-uploading"). Its output $f(x) = \langle Z \rangle$ is a trigonometric series in x. The highest frequency it can contain is

(A) 8 **(B)** ✓ 3 **(C)** 1 **(D)** 6

Answer: (B) Detailed solution

Correct answer. (B) 3
Concepts required. Frequency spectrum of quantum models; data re-uploading.
Relevant formulae. $f(x) = \sum_{\omega \in \Omega} c_\omega e^{i\omega x}$ with $\Omega = \{\lambda_j - \lambda_k\}$ of the sums of generator eigenvalues; L uses of $e^{-ixY/2}$: $\Omega = \{-L, \ldots, L\}$.
Step-by-step explanation. Each $R_Y(x) = e^{-ixY/2}$ has generator eigenvalues $\pm\frac{1}{2}$. In $f = \langle\psi|M|\psi\rangle$ each occurrence contributes phases $e^{\pm ix/2}$ from the ket and from the bra; with L occurrences the exponents are differences of sums of L terms $\pm\frac{1}{2}$, so the frequencies are the integers $0, 1, \ldots, L$ (Schuld, Sweke & Meyer 2021). With one encoding gate $f(x) = a + b\cos x + c\sin x$; with $L = 3$ the spectrum reaches frequency 3. Repeating the encoding increases expressivity.
Why the other options are incorrect.
(A) $8 = 2^3$ confuses frequencies with the number of basis states.
(C) Frequency 1 is the single-encoding case $L = 1$.
(D) 6 double-counts: the ket and the bra together give differences of sums of $\pm\frac{1}{2}$, reaching at most L.
Key takeaway. L encodings ⇒ Fourier series of degree L in the data.

Q.55 MATCHING LISTS · Encodings and feature maps

Match each data encoding (List-I) with its description (List-II); d = number of features.

List-I		List-II	
(P)	basis encoding of a d-bit string	**(1)**	$\log_2 d$ qubits; preparing a general state needs $O(d)$ gates
(Q)	angle encoding $\bigotimes_i R_Y(x_i)\,\|0\rangle$	**(2)**	d qubits; an X gate on every qubit whose bit is 1; basis states only
(R)	amplitude encoding of $d = 2^n$ real numbers	**(3)**	d qubits; Hadamards, then phases $e^{-i\phi_i(\mathbf{x})Z_i}$ and $e^{-i\phi_{ij}(\mathbf{x})Z_iZ_j}$; entangling; conjectured hard to simulate classically
(S)	ZZ (IQP-type) feature map	**(4)**	d qubits; depth 1; product state, so its kernel factorises

(A) P→1, Q→4, R→2, S→3 **(B)** P→4, Q→2, R→1, S→3

(C) ✓ P→2, Q→4, R→1, S→3 **(D)** P→2, Q→4, R→3, S→1

Answer: (C) Detailed solution

Correct answer. (C) P→2, Q→4, R→1, S→3

Concepts required. The four standard ways to put classical data into qubits.

Relevant formulae. Basis $|b\rangle$; angle $\bigotimes R(x_i)\,|0\rangle$; amplitude $\sum x_j\,|j\rangle\,/\|\mathbf{x}\|$; ZZ map $U_\Phi(\mathbf{x})H^{\otimes n}U_\Phi(\mathbf{x})H^{\otimes n}$.

Step-by-step explanation.

- Basis encoding: $|b_1 \dots b_d\rangle$ prepared by X gates (2).
- Angle encoding (used by `QML.ipynb` with angles πx_i): one rotation per qubit, no entanglement (4).
- Amplitude encoding: exponentially few qubits, but generic state preparation is expensive (1).
- The ZZ feature map of Havlíček et al. (2019) uses entangling Z_iZ_j phases; its kernel is believed to be hard to compute classically for suitable parameters (3).

Hence P→2, Q→4, R→1, S→3.

Why the other options are incorrect.

(A) P→1, R→2 gives basis encoding $\log_2 d$ qubits; it needs one qubit per bit.

(B) P→4, Q→2 swaps basis and angle encoding: angle encoding uses rotations, not X gates.

(D) R→3, S→1 swaps amplitude encoding and the ZZ map: the ZZ map uses d qubits and entangling phases.

Key takeaway. Basis: bits; angle: rotations (`QML.ipynb`); amplitude: normalised vector; ZZ: entangling feature map.

Q.56 MATCHING LISTS · What each QML model trains

Match each QML model (List-I) with what is trained (List-II).

List-I		List-II	
(P)	quantum-kernel SVM	**(1)**	a parameterised generator circuit, trained adversarially against a discriminator
(Q)	variational quantum classifier (VQC)	**(2)**	no circuit parameters: the feature map is fixed; the classical dual coefficients α_m are trained
(R)	QCNN	**(3)**	a feature map followed by a trainable ansatz $U(\boldsymbol{\theta})$; $\boldsymbol{\theta}$ is updated by a classical optimizer
(S)	quantum GAN	**(4)**	convolution and pooling layers with shared parameters; $O(\log N)$ parameters; one output qubit

(A) P→3, Q→2, R→4, S→1 **(B)** P→2, Q→3, R→1, S→4

(C) ✓ P→2, Q→3, R→4, S→1 **(D)** P→2, Q→4, R→3, S→1

Answer: (C) Detailed solution

Correct answer. (C) P→2, Q→3, R→4, S→1

Concepts required. Where the trainable parameters live in different QML models.

Relevant formulae. Kernel SVM: $\hat{y} = \operatorname{sign}\sum_m \alpha_m y_m k(\mathbf{x}_m, \mathbf{x}) + b$; VQC: $f = \langle\psi(\mathbf{x})|U^\dagger(\boldsymbol{\theta})MU(\boldsymbol{\theta})|\psi(\mathbf{x})\rangle$.

Step-by-step explanation.

- A kernel SVM uses the quantum computer only to estimate $k(\mathbf{x}_i, \mathbf{x}_j)$; training is a convex classical problem for α (2).
- A VQC trains the circuit parameters in a hybrid loop (3).
- The QCNN of `QML.ipynb`: 3 layers × 4 shared parameters on 8 qubits, readout $\langle Z_7\rangle$ (4).
- A quantum GAN trains a generator circuit to fool a (classical or quantum) discriminator (1).

Hence P→2, Q→3, R→4, S→1.

Why the other options are incorrect.

(A) P→3, Q→2 swaps kernel SVM and VQC: a kernel SVM does not train circuit parameters.

(B) R→1, S→4 swaps QCNN and GAN: adversarial training defines the GAN.

(D) Q→4, R→3 swaps VQC and QCNN: the QCNN's defining feature is shared conv/pool layers.

Key takeaway. Kernel methods: quantum kernel, classical training; variational models: trainable circuits.

Q.57 Assertion–Reason · Gradient-free training; shift-rule validity

Assertion (A): `QML.ipynb` trains its QCNN without evaluating any gradient circuits.

Reason (R): Several QCNN parameters enter controlled rotations (`rz.ctrl`, `rx.ctrl`), for which the two-term parameter-shift rule with shifts $\pm\frac{\pi}{2}$ is not exact.

(A) Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A

(B) ✓ Both A and R are true, but R is *not* the correct explanation of A

(C) A is true, but R is false

(D) A is false, but R is true

Answer: (B) Detailed solution

Correct answer. **(B)** Both A and R are true, but R is *not* the correct explanation of A

Concepts required. Derivative-free optimizers vs. gradient rules; validity of the two-term shift rule.

Relevant formulae. Two-term rule exact iff the generator has exactly two eigenvalues $\pm r$; controlled rotations have $\{0, 0, \pm\frac{1}{2}\}$.

Step-by-step explanation. **A is true:** the default optimizer is COBYLA (Nelder–Mead and Powell are the alternatives); all three are derivative-free — they only call `loss(theta)`. **R is true:** the generator of a controlled rotation, $|1\rangle\langle 1| \otimes \frac{P}{2}$, has eigenvalues $0, \pm\frac{1}{2}$, so f contains the frequencies $\frac{1}{2}$ and 1 and needs a four-term shift rule. But R is *not* the reason for A: the notebook would use COBYLA even for a circuit with only R_Y gates; the choice of a gradient-free optimizer is a design decision, not forced by the gate set. Answer: (B).

Why the other options are incorrect.

(A) needs R to explain A; but COBYLA is chosen for robustness and simplicity, not because shift rules fail.

(C) needs R false; the generator of a controlled rotation has three distinct eigenvalues, so R is true.

(D) needs A false; COBYLA, Nelder–Mead and Powell never evaluate gradients.

Key takeaway. Two true statements are not automatically cause and effect.

Q.58 Statement I / II · PSD kernel matrices; shot noise

Statement I: For any feature map, the exact Gram matrix $K_{ij} = |\langle\psi(\mathbf{x}_i)|\psi(\mathbf{x}_j)\rangle|^2$ is symmetric and positive semi-definite.

Statement II: When the entries K_{ij} are *estimated* with a finite number of shots, the estimated matrix can have small negative eigenvalues.

(A) ✓ Both Statement I and Statement II are true

(B) Both Statement I and Statement II are false

(C) Statement I is true, but Statement II is false

(D) Statement I is false, but Statement II is true

Answer: (A) Detailed solution

Correct answer. **(A)** Both Statement I and Statement II are true

Concepts required. Positive semi-definiteness of kernel matrices; effect of finite sampling.

Relevant formulae. $\sum_{ij} c_i c_j K_{ij} = \mathrm{tr}(A^2) \geq 0$, $A = \sum_i c_i \rho_i$ Hermitian.

Step-by-step explanation. **I is true:** $|\langle\psi_i|\psi_j\rangle|^2 = \mathrm{tr}(\rho_i\rho_j)$ with $\rho_i = |\psi_i\rangle\langle\psi_i|$, a Hilbert–Schmidt inner product, so $\sum_{ij} c_i c_j K_{ij} = \mathrm{tr}\left[(\sum_i c_i\rho_i)^2\right] \geq 0$ for real c. **II is true:** each estimate carries independent statistical noise of order $1/\sqrt{S}$; the perturbed symmetric matrix need not be PSD, especially when M is large and the true smallest eigenvalues are near 0. In practice the spectrum is clipped or a small multiple of I is added. Answer: (A).

Why the other options are incorrect.

(B) needs both false, but I is a theorem and II is observed in practice.

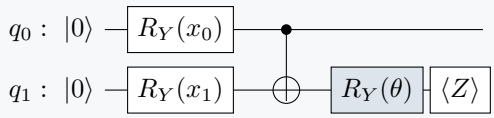
(C) needs II false, but shot noise perturbs every entry independently and can break PSD.

(D) needs I false, but every fidelity kernel is a valid (PSD) kernel.

Key takeaway. Exact quantum kernels are valid kernels; estimated ones may need regularisation.

Paragraph for Questions 59 and 60

A two-qubit variational quantum classifier (VQC) encodes the features (x_0, x_1) with $R_Y(x_0)$ on qubit 0 and $R_Y(x_1)$ on qubit 1, applies a CNOT (control 0, target 1) and then one trainable gate $R_Y(\theta)$ on qubit 1. Its output is $f(\mathbf{x};\theta) = \langle Z_1 \rangle$, the prediction is $\operatorname{sign} f$, and the loss of one example is $L = (f - y)^2$.



Q.59 Paragraph based · VQC output (Heisenberg picture)

The model output $f(\mathbf{x};\theta) = \langle Z_1 \rangle$ equals

(A) $\cos\theta\cos x_1 - \sin\theta\sin x_0\sin x_1$

(B) ✓ $\cos\theta\cos x_0\cos x_1 - \sin\theta\sin x_1$

(C) $\cos\theta\cos x_0\cos x_1 + \sin\theta\sin x_1$

(D) $\cos(\theta + x_1)\cos x_0$

Answer: (B) Detailed solution

Correct answer. **(B)** $\cos\theta\cos x_0\cos x_1 - \sin\theta\sin x_1$

Concepts required. Heisenberg picture for a small VQC; CNOT propagation rules.

Relevant formulae. $R_Y^\dagger Z R_Y = \cos\theta Z - \sin\theta X$; CNOT: $Z_t \to Z_c Z_t$, $X_t \to X_t$; $\langle X \rangle = \sin x$ for $R_Y(x)\,|0\rangle$.

Step-by-step explanation. Work backwards with the observable (Heisenberg picture). (1) Through $R_Y(\theta)$ on qubit 1: $Z_1 \to \cos\theta\, Z_1 - \sin\theta\, X_1$ (from $R_Y^\dagger Z R_Y = \cos\theta Z - \sin\theta X$). (2) Through the CNOT: Z on the target becomes $Z_0 Z_1$, while X on the target is unchanged. (3) On the product state $R_Y(x_0)\,|0\rangle \otimes R_Y(x_1)\,|0\rangle$: $\langle Z_0 \rangle = \cos x_0$, $\langle Z_1 \rangle = \cos x_1$, $\langle X_1 \rangle = \sin x_1$. Hence $f = \cos\theta\cos x_0\cos x_1 - \sin\theta\sin x_1$.

Why the other options are incorrect.

(A) This attaches $\cos x_0$ to the wrong term and turns $\langle X_1 \rangle$ into $\sin x_0 \sin x_1$.

(C) $+\sin\theta\sin x_1$ has the sign of the conjugation wrong.

(D) $\cos(\theta + x_1)\cos x_0$ attaches $\cos x_0$ to the X_1 term too; X on the target does not pick up Z_0.

Key takeaway. Push the observable back through the gates, then evaluate on the product state.

Q.60 PARAGRAPH BASED · VQC gradient by parameter shift

For $\mathbf{x} = (0, \frac{\pi}{3})$, $\theta = \frac{\pi}{6}$ and label $y = +1$, the shifted outputs $f(\theta + \frac{\pi}{2})$, $f(\theta - \frac{\pi}{2})$ and the loss gradient $\partial L/\partial\theta$ are

(A) $+1,\ -1,\ +2$ **(B)** $-1,\ +1,\ -2$ **(C)** $-1,\ +1,\ -1$ **(D)** ✓ $-1,\ +1,\ +2$

Answer: (D) Detailed solution

Correct answer. (D) $-1,\ +1,\ +2$

Concepts required. Parameter shift + chain rule = gradient of the training loss.

Relevant formulae. $f' = \frac{1}{2}[f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})]$; $\frac{\partial L}{\partial\theta} = 2(f - y)f'$.

Step-by-step explanation. With $x_0 = 0$, $x_1 = \frac{\pi}{3}$: $f(\theta) = \frac{1}{2}\cos\theta - \frac{\sqrt{3}}{2}\sin\theta = \cos(\theta + \frac{\pi}{3})$. $f(\frac{\pi}{6}) = \cos\frac{\pi}{2} = 0$; $f(\frac{\pi}{6} + \frac{\pi}{2}) = f(\frac{2\pi}{3}) = \cos(\frac{2\pi}{3} + \frac{\pi}{3}) = \cos\pi = -1$; $f(\frac{\pi}{6} - \frac{\pi}{2}) = f(-\frac{\pi}{3}) = \cos(-\frac{\pi}{3} + \frac{\pi}{3}) = 1$. Parameter shift: $f'(\theta) = \frac{1}{2}(-1 - 1) = -1$ (exact; check $f' = -\sin(\theta + \frac{\pi}{3}) = -1$). Chain rule: $\frac{\partial L}{\partial\theta} = 2(f - y)f' = 2(0 - 1)(-1) = +2$. A gradient step $\theta \leftarrow \theta - \eta \cdot 2$ decreases θ, which increases f towards the label $+1$.

Why the other options are incorrect.

(A) The shifted values are swapped: $f(\frac{2\pi}{3}) = \cos\pi = -1$, $f(-\frac{\pi}{3}) = \cos 0 = +1$.

(B) -2 has the sign of $\partial L/\partial\theta$ wrong: $2(f - y)f' = 2(-1)(-1) = +2$.

(C) -1 is $f'(\theta)$, the derivative of the *output*, not of the loss.

Key takeaway. Quantum part: f' by parameter shift; classical part: chain rule through the loss.

SILICO FELLER

SECTION D · Advanced QML and Implementation Q.61–80

Q.61 Single-correct MCQ · Barren plateau: shots needed

For a deep random circuit with a global cost function the gradient components satisfy $\mathbb{E}[\partial_k C] = 0$ and $\mathrm{Var}[\partial_k C] \approx 2^{-n}$. Each cost value is estimated with S shots (statistical error $\approx 1/\sqrt{S}$). To resolve a typical gradient component at $n = 20$ qubits one needs roughly

(A) $S \sim 20$ shots **(B)** ✓ $S \sim 10^6$ shots **(C)** $S \sim 10^{12}$ shots **(D)** $S \sim 400$ shots

Answer: (B) Detailed solution

Correct answer. (B) $S \sim 10^6$ shots
Concepts required. Barren plateaus make gradients exponentially small; estimating them needs exponentially many shots.
Relevant formulae. $\mathrm{Var}[\partial_k C] \sim 2^{-n}$; shot error $\sim 1/\sqrt{S}$; need $1/\sqrt{S} \lesssim 2^{-n/2}$.
Step-by-step explanation. A typical gradient has size $\sqrt{\mathrm{Var}} = 2^{-n/2}$ (by Chebyshev, $|\partial_k C| \geq \epsilon$ has probability $\leq 2^{-n}/\epsilon^2$). To see it above the shot noise we need $1/\sqrt{S} \lesssim 2^{-n/2}$, i.e. $S \gtrsim 2^n = 2^{20} \approx 1.05 \times 10^6$ — *exponentially* many shots in the number of qubits: a barren plateau. Comparing the noise with the variance instead of the standard deviation would give $2^{2n} \approx 10^{12}$; polynomial guesses (n, n^2) miss the exponential scaling.
Why the other options are incorrect.
(A) 20 shots $(= n)$ assumes polynomial scaling, ignoring the 2^{-n} variance.
(C) $10^{12} = 2^{2n}$ compares the shot error with the variance instead of the standard deviation.
(D) $400 = n^2$ is also polynomial; it misses the exponential.
Key takeaway. Shots $\sim 2^n$: at 20 qubits about a million per gradient component.

Q.62 Single-correct MCQ · Barren plateaus: what is known

Which statement about barren plateaus is correct?

(A) ✓ For random circuits deep enough to form approximate 2-designs, gradient variances vanish exponentially in n; for shallow circuits, local cost functions have only polynomially vanishing gradients while global cost functions can still vanish exponentially.

(B) Barren plateaus are caused only by hardware noise, so they never appear in noiseless simulation.

(C) Switching to a gradient-free optimizer such as COBYLA removes a barren plateau, because no gradients are computed.

(D) A barren plateau can be removed by multiplying the learning rate by 2^n.

Answer: (A) Detailed solution

Correct answer. (A) For random circuits deep enough to form approximate 2-designs, gradient variances vanish exponentially in n; for shallow circuits, local cost functions have only polynomially vanishing gradients while global cost functions can still vanish exponentially.
Concepts required. Known results about barren plateaus: depth, cost locality, noise, optimizer independence.
Relevant formulae. 2-design: $\mathrm{Var}[\partial C] \in O(2^{-n})$; shallow + local cost: $\Omega(1/\mathrm{poly}(n))$; global cost: $O(2^{-n})$.
Step-by-step explanation. McClean et al. (2018) showed exponentially vanishing gradient variance for random parameterised circuits that approximate 2-designs; Cerezo et al. (2021) showed that for shallow (logarithmic-depth) circuits the *locality* of the cost matters: local costs give polynomially vanishing gradients, global ones (e.g. fidelity with a target state) exponentially vanishing ones. Noise

can *cause* additional plateaus (Wang et al. 2021), but noiseless deep circuits already have them. A flat landscape is flat for every optimizer: cost *differences* are exponentially small too (Arrasmith et al. 2021). Rescaling the step does not help because the gradient estimate is dominated by shot noise.

Why the other options are incorrect.

(B) Noiseless random circuits already have barren plateaus (McClean et al.); noise adds further ones.

(C) Gradient-free optimizers also need cost differences, which are exponentially small on a plateau.

(D) The gradient *estimate* is shot-noise dominated; scaling the step amplifies the noise too.

Key takeaway. Avoid plateaus by structure: shallow circuits, local costs, symmetric architectures (e.g. QCNN), smart initialisation.

Q.63 SINGLE-CORRECT MCQ · Two- vs. four-term shift rule

In a Hadamard test with a controlled-$R_Z(\theta)$ the measured output is $f(\theta) = \cos(\theta/2)$ (a pure frequency $\frac{1}{2}$). At $\theta = \pi$, the two-term shift formula $\frac{1}{2}[f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})]$ and the exact derivative $f'(\pi)$ are

(A) ✓ -0.707 and -0.5

(B) -0.5 and -0.5

(C) -1 and -0.5

(D) -0.707 and -0.707

Answer: (A) Detailed solution

Correct answer. (A) -0.707 and -0.5

Concepts required. Generalised parameter-shift rules for generators with more than two eigenvalues.

Relevant formulae. Two-term: exact for frequency 1; four-term: $d_\pm = \frac{\sqrt{2}\pm 1}{4\sqrt{2}}$, shifts $\frac{\pi}{2}$ and $\frac{3\pi}{2}$.

Step-by-step explanation. Exact: $f'(\theta) = -\frac{1}{2}\sin\frac{\theta}{2}$, so $f'(\pi) = -\frac{1}{2}$. Two-term formula: $\frac{1}{2}[\cos\frac{3\pi}{4} - \cos\frac{\pi}{4}] = \frac{1}{2}[-\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}] = -\frac{\sqrt{2}}{2} \approx -0.707$ — wrong by 41 %. The two-term rule is exact only when the generator has two eigenvalues $\pm\frac{1}{2}$ (frequency 1). A controlled rotation has eigenvalues $0, \pm\frac{1}{2}$ (frequencies $\frac{1}{2}$ and 1); the four-term rule $f' = d_+[f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})] - d_-[f(\theta + \frac{3\pi}{2}) - f(\theta - \frac{3\pi}{2})]$, $d_\pm = \frac{\sqrt{2}\pm 1}{4\sqrt{2}}$, gives exactly -0.5.

Why the other options are incorrect.

(B) -0.5 for the two-term formula assumes it is exact; it is not for frequency $\frac{1}{2}$.

(C) -1 omits the factor $\frac{1}{2}$ of the shift formula.

(D) -0.707 is not the exact derivative; $\frac{d}{d\theta}\cos\frac{\theta}{2} = -\frac{1}{2}\sin\frac{\theta}{2} = -\frac{1}{2}$ at π.

Key takeaway. Controlled rotations need the four-term rule; the naive two-term rule is biased.

Q.64 SINGLE-CORRECT MCQ · Depolarising noise and accuracy

In a simple global-depolarising model every gate multiplies the output expectation value by $(1 - p)$. For the default QCNN (69 gates) and $p = 0.01$, the expectation values are multiplied by about ..., and the test *accuracy* (prediction $\operatorname{sign}\langle Z\rangle$) is expected to

(A) 0.31; drop to 50 %

(B) ✓ 0.50; stay the same, while $|\langle Z\rangle|$ shrinks and the MSE grows

(C) 0.50; drop to about 50 %, because half of the predictions flip sign

(D) 0.99; stay the same, and noise is negligible

Answer: (B) Detailed solution

Correct answer. **(B)** 0.50; stay the same, while $|\langle Z\rangle|$ shrinks and the MSE grows
Concepts required. Depolarising noise shrinks expectation values towards 0 without (on average) flipping their sign.
Relevant formulae. $\langle Z\rangle_{\text{noisy}} \approx (1-p)^G\langle Z\rangle$; $(1-p)^G \approx e^{-pG}$ for small p.
Step-by-step explanation. $(1-0.01)^{69} = e^{69\ln 0.99} = e^{-0.6935} \approx 0.50$. Multiplying every output by a positive factor does not change its sign, so sign-based accuracy is unchanged in this model; but the outputs move towards 0, so the MSE $\frac{1}{M}\sum(f-y)^2$ grows towards 1 and, with finite shots, small outputs become indistinguishable from 0. (The lecture's density-matrix simulation with gate-level depolarising noise $p = 0.01$ found exactly this: accuracy stayed 85 % while the mean $|\langle Z\rangle|$ fell from 0.171 to 0.112.)
Why the other options are incorrect.
(A) 0.31 uses e^{-pG} with the wrong exponent; and a positive scale factor cannot flip signs.
(C) A common positive factor keeps every sign; accuracy does not collapse in this model.
(D) 0.99 is one gate's factor; 69 gates compound to about 0.5.
Key takeaway. Noise shrinks margins (and raises MSE) before it flips predictions; small margins are fragile.

Q.65 Single-correct MCQ · GPU memory and multi-GPU qubits

An 80 GB GPU simulates state vectors in double precision (16 bytes per amplitude). The largest number of qubits that fits on one GPU, and on four such GPUs pooled for one distributed state vector, are

(A) 36 and 38 **(B)** 32 and 128 **(C)** 33 and 35 **(D)** ✓ 32 and 34

Answer: (D) Detailed solution

Correct answer. **(D)** 32 and 34
Concepts required. GPU memory limits of state-vector simulation; distributed (multi-GPU) state vectors.
Relevant formulae. $n_{\max} = \lfloor \log_2(\text{memory}/b) \rfloor$, $b = 16$ (fp64) or 8 (fp32).
Step-by-step explanation. Need $16 \cdot 2^n \le 80 \times 10^9$: $2^{32}\cdot 16 = 68.7$ GB fits, $2^{33}\cdot 16 = 137$ GB does not, so $n = 32$. Four GPUs give 320 GB: $2^{34}\cdot 16 = 275$ GB fits, $2^{35}\cdot 16 = 550$ GB does not, so $n = 34$. Multiplying the memory by 4 adds only $\log_2 4 = 2$ qubits. (In single precision, 8 bytes, the answers would be 33 and 35.)
Why the other options are incorrect.
(A) 36 and 38 divide bytes as if they were bits.
(B) 128 scales qubits linearly with memory; memory scales as 2^n.
(C) 33 and 35 are the single-precision (8-byte) answers.
Key takeaway. 4× the memory buys only 2 more qubits.

Q.66 SINGLE-CORRECT MCQ · Cost of an exact QCNN gradient

Suppose the 8-qubit QCNN (11 convolution blocks, 7 pooling blocks) were trained by gradient descent with *exact* parameter-shift gradients: every R_Y occurrence shifted separately (2 circuits), every controlled rotation (`crz`, `crx`) with the four-term rule (4 circuits). The number of circuits for *one* full gradient of the loss over the 20 training images is

(A) 480 **(B)** 1440 **(C)** 40 **(D)** ✓ 2000

Answer: (D) Detailed solution

Correct answer. (D) 2000

Concepts required. Cost of exact gradients for circuits with shared parameters and controlled rotations.

Relevant formulae. Per image: $\sum_{\text{conv}} 2 \times 2 + \sum_{\text{pool}} 2 \times 4 = 4n_{\text{conv}} + 8n_{\text{pool}}$.

Step-by-step explanation. Per image: each convolution block has 2 R_Y gates $\Rightarrow 2 \times 2 = 4$ circuits per block, $11 \times 4 = 44$; each pooling block has one `crz` and one `crx` $\Rightarrow 2 \times 4 = 8$ circuits per block, $7 \times 8 = 56$. Total $44 + 56 = 100$ circuits per image, and $100 \times 20 = 2000$ for the training set (plus the 20 forward passes for the loss). One exact gradient step costs as much as the *whole* 100-evaluation COBYLA run of the notebook. (480 = naive $2 \times 12 \times 20$; 1440 = two-term rule for every occurrence.)

Why the other options are incorrect.

(A) $480 = 2 \times 12 \times 20$ shifts each shared parameter once with the two-term rule: wrong for shared and controlled gates.

(B) 1440 shifts every occurrence but uses the two-term rule for the controlled rotations, which is not exact.

(C) 40 counts two circuits per image only.

Key takeaway. Exact gradients are expensive: one step costs as much as the whole COBYLA run here.

Q.67 SINGLE-CORRECT MCQ · Expectation values from counts

`cudaq.sample` on a two-qubit kernel returns the counts `{'00': 400, '01': 100, '10': 300, '11': 200}` (the first character is qubit 0). The estimates of $(\langle Z_0\rangle, \langle Z_1\rangle, \langle Z_0Z_1\rangle)$ are

(A) ✓ (0, 0.4, 0.2) **(B)** (0, 0.4, 0) **(C)** (0, 0.4, 0.6) **(D)** (0.4, 0, 0.2)

Answer: (A) Detailed solution

Correct answer. (A) (0, 0.4, 0.2)

Concepts required. Estimating Pauli expectation values from bitstring counts; CUDA-Q writes qubit 0 first.

Relevant formulae. $\langle Z_i\rangle = \sum_s p_s(-1)^{s_i}$, $\langle Z_0Z_1\rangle = \sum_s p_s(-1)^{s_0+s_1}$.

Step-by-step explanation. Total 1000 shots. Qubit 0 is 0 in `00`, `01`: $P = 0.5$, so $\langle Z_0\rangle = 0.5 - 0.5 = 0$. Qubit 1 is 0 in `00`, `10`: $P = 0.7$, so $\langle Z_1\rangle = 0.7 - 0.3 = 0.4$. $Z_0Z_1 = +1$ for even parity (`00`, `11`: 600) and -1 for odd parity (400): $\langle Z_0Z_1\rangle = 0.2$, which differs from $\langle Z_0\rangle\langle Z_1\rangle = 0$, so the qubits are correlated. `cudaq.observe` computes such expectation values directly (exactly on a state-vector simulator, or from shots with `shots_count`).

Why the other options are incorrect.

(B) $\langle Z_0Z_1\rangle = 0$ multiplies the marginals; the parity counts give 0.2.

(C) 0.6 is P(even parity), not P(even) − P(odd).

(D) (0.4, 0) reads the bitstrings in the wrong order (last character = qubit 0).

Key takeaway. Marginalise the counts per qubit; parity gives correlators.

Q.68 Single-correct MCQ · Interpreting MSE 0.67 at 90 % accuracy

The recorded run of `QML.ipynb` ends with training accuracy 90 % but training MSE 0.67 (the mean $|\langle Z_7\rangle|$ of the trained model on the training images is about 0.20). The best explanation is

(A) ✓ the outputs have the correct sign but small magnitude, so each correct image still contributes about $(1-0.2)^2 = 0.64$ to the loss

(B) a bug: for ± 1 labels the MSE must equal $1 - \text{accuracy} = 0.10$

(C) COBYLA diverged in the last iterations, so the loss and the accuracy belong to different parameter vectors

(D) the MSE is computed on the test images, the accuracy on the training images

Answer: (A) Detailed solution

Correct answer. **(A)** the outputs have the correct sign but small magnitude, so each correct image still contributes about $(1-0.2)^2 = 0.64$ to the loss
Concepts required. MSE vs. accuracy; output margins of a trained QCNN.
Relevant formulae. For a correct prediction with $|f| = m$: $(f-y)^2 = (1-m)^2$.
Step-by-step explanation. MSE measures the distance of the real-valued output from ± 1; accuracy only checks the sign. If $\langle Z\rangle = \pm 0.2$ with the right sign, the squared error is $(1-0.2)^2 = 0.64$ even though the prediction is correct. A mean $|\langle Z\rangle|$ of 0.2 is consistent with an MSE of 0.67 and 90 % accuracy. The notebook prints both quantities for the same final parameters on the training set. Small margins matter: with few shots or noise such predictions are fragile.
Why the other options are incorrect.

(B) For outputs in $[-1, 1]$ there is no identity MSE $= 1 -$ accuracy; that would hold only for outputs exactly ± 1 (then MSE $= 4(1 - \text{acc})$).

(C) COBYLA returns its best point; the notebook recomputes the loss and accuracy at the same final parameters.

(D) Both are computed on the training set (the test MSE is not printed).

Key takeaway. High accuracy with high MSE means correct but low-confidence outputs.

Q.69 Single-correct MCQ · 15-qubit training time

The README of the repository reports a 15-qubit test of `QML.ipynb` (3×5 images, 16 training images, 60 COBYLA evaluations) at about 0.35 s per circuit on the 4-core test CPU. The training time is about

(A) 21 seconds **(B)** ✓ 5.6 minutes **(C)** 0.35 seconds **(D)** 56 minutes

Answer: (B) Detailed solution

Correct answer. **(B)** 5.6 minutes
Concepts required. Training cost = evaluations × images × time per circuit.
Relevant formulae. $T \approx n_{\text{eval}} \cdot M \cdot t_{\text{circuit}}$.
Step-by-step explanation. Circuits = 60 evaluations × 16 images = 960; time $\approx 960 \times 0.35$ s = 336 s $\approx$ 5.6 minutes. (Compare 8 qubits: 1.7 ms per circuit. Going from 8 to 15 qubits multiplies the state size by $2^7 = 128$ and the gate count from 69 to 146, so a factor of about 200 is expected.) This is why the notebook recommends a GPU for 15 or more qubits.
Why the other options are incorrect.

(A) 21 s forgets the 16 images per evaluation.

(C) 0.35 s is the time of one circuit.

(D) 56 min is ten times too large.

Key takeaway. $60 \times 16 \times 0.35$ s $\approx$ 5.6 min on a CPU: GPU recommended beyond $\sim$ 15 qubits.

Q.70 SINGLE-CORRECT MCQ · Kernel concentration

For the product kernel $k(\mathbf{x},\mathbf{x}') = \prod_{i=1}^{n} \cos^2 \frac{\pi(x_i - x_i')}{2}$ with all x_i, x_i' independent and uniform on $[0,1]$, the average off-diagonal kernel value $\mathbb{E}[k]$ equals

(A) $(2/\pi)^n$ **(B)** 2^{-n}

(C) ✓ $\left(\frac{1}{2} + \frac{2}{\pi^2}\right)^n \approx 0.70^n$ **(D)** $1/n$

Answer: (C) Detailed solution

Correct answer. (C) $\left(\frac{1}{2} + \frac{2}{\pi^2}\right)^n \approx 0.70^n$

Concepts required. Exponential concentration of kernel values for product feature maps with many features.

Relevant formulae. $\mathbb{E}[k] = \prod_i \mathbb{E}\cos^2 \frac{\pi\Delta_i}{2}$; $\mathbb{E}\cos\pi(x-x') = |\mathbb{E}e^{i\pi x}|^2 = 4/\pi^2$.

Step-by-step explanation. Independence factorises the expectation: $\mathbb{E}[k] = (\mathbb{E}\cos^2\frac{\pi\Delta}{2})^n$, $\Delta = x - x'$. $\cos^2\frac{\pi\Delta}{2} = \frac{1}{2} + \frac{1}{2}\cos\pi\Delta$ and $\mathbb{E}\cos\pi(x-x') = |\mathbb{E}\,e^{i\pi x}|^2 = \left|\int_0^1 e^{i\pi x}dx\right|^2 = \left|\frac{e^{i\pi}-1}{i\pi}\right|^2 = \frac{4}{\pi^2}$. So each factor is $\frac{1}{2} + \frac{2}{\pi^2} \approx 0.703$ and $\mathbb{E}[k] \approx 0.703^n$: kernel values *concentrate* exponentially towards 0 as the number of features grows ($0.703^8 \approx 0.06$, $0.703^{50} \approx 2\times10^{-8}$), so exponentially many shots would be needed to estimate them — the exponential-concentration problem of quantum kernels.

Why the other options are incorrect.

(A) $(2/\pi)^n$ uses $|\mathbb{E}e^{i\pi x}|$ instead of $\frac{1}{2} + \frac{1}{2}|\mathbb{E}e^{i\pi x}|^2$.

(B) 2^{-n} assumes every factor averages $\frac{1}{2}$, i.e. drops the term $\frac{1}{2}\mathbb{E}\cos\pi\Delta = 2/\pi^2$.

(D) $1/n$ is polynomial; independent factors multiply, giving exponential decay.

Key takeaway. Product kernels concentrate like c^n ($c \approx 0.70$ here): more features ⇒ kernel values $\to 0$.

Q.71 SINGLE-CORRECT MCQ · Quantum PCA

Which statement about quantum principal component analysis (qPCA, Lloyd, Mohseni & Rebentrost 2014) is correct?

(A) ✓ It uses many copies of a density matrix ρ to apply $e^{-i\rho t}$ (density-matrix exponentiation) and phase estimation to reveal its large eigenvalues; the output is a quantum state, and any speed-up assumes efficient state preparation and low rank — dequantized classical algorithms match it under comparable assumptions.

(B) It is the pooling layer of the QCNN in `QML.ipynb`.

(C) It diagonalises the covariance matrix of N classical data points in $O(\log N)$ total time, including loading the data and reading out all eigenvectors.

(D) It needs only a single copy of ρ, because one measurement reveals all eigenvalues.

Answer: (A) Detailed solution

Correct answer. (A) It uses many copies of a density matrix ρ to apply $e^{-i\rho t}$ (density-matrix exponentiation) and phase estimation to reveal its large eigenvalues; the output is a quantum state, and any speed-up assumes efficient state preparation and low rank — dequantized classical algorithms match it under comparable assumptions.

Concepts required. What qPCA actually does, and its assumptions.

Relevant formulae. $e^{-i\rho t}$ from copies of ρ (partial SWAPs); phase estimation gives λ_k with $|v_k\rangle$; cost polylog only for low rank with efficient state access.

Step-by-step explanation. qPCA prepares many copies of ρ (the normalised covariance/density matrix), simulates $e^{-i\rho t}$ by repeated partial SWAPs with fresh copies, and runs phase estimation to obtain eigenvalue estimates together with the eigenvectors *as quantum states*. Reading out classical eigenvectors costs time proportional to their dimension, and loading classical data needs QRAM-like assumptions; Tang-style "dequantized" sampling algorithms reach similar polylogarithmic scaling

for low-rank matrices with comparable access. One copy cannot reveal a spectrum (no-cloning, measurement collapse). qPCA is *not* implemented in this repository.

Why the other options are incorrect.

(B) The QCNN pooling block is a controlled rotation; qPCA is not in the repository.

(C) Loading N classical points and reading out eigenvectors each cost at least linear time; no end-to-end $O(\log N)$ is known.

(D) One copy of an unknown state cannot reveal its spectrum; qPCA consumes many copies.

Key takeaway. qPCA's speed-up lives in the assumptions (state access, low rank, quantum output).

Q.72 SINGLE-CORRECT MCQ · GAN equilibrium

A (quantum or classical) GAN optimises $V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\ln D(x)] + \mathbb{E}_{x\sim p_G}[\ln(1-D(x))]$. At the ideal equilibrium $p_G = p_{\text{data}}$ with the optimal discriminator, the value of V is

(A) 0 **(B)** $-\frac{1}{2}$ **(C)** ✓ $-\ln 4 \approx -1.386$ **(D)** $\ln 2$

Answer: (C) Detailed solution

Correct answer. (C) $-\ln 4 \approx -1.386$

Concepts required. GAN minimax objective and its equilibrium.

Relevant formulae. $D^* = \frac{p_{\text{data}}}{p_{\text{data}}+p_G}$; at $p_G = p_{\text{data}}$: $D^* = \frac{1}{2}$, $V = 2\ln\frac{1}{2}$.

Step-by-step explanation. For fixed G the optimal discriminator is $D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x)+p_G(x)}$. When $p_G = p_{\text{data}}$, $D^* = \frac{1}{2}$ everywhere, so $V = \ln\frac{1}{2} + \ln\frac{1}{2} = -2\ln 2 = -\ln 4$. In a quantum GAN the generator is a parameterised circuit whose measurement distribution plays the role of p_G; training alternates discriminator and generator updates (not implemented in this repository).

Why the other options are incorrect.

(A) 0 would need $D = 1$ on data and $D = 0$ on fakes, i.e. a perfect discriminator, not equilibrium.

(B) $-\frac{1}{2}$ has no derivation from V.

(D) $\ln 2$ is positive; V is a sum of logarithms of probabilities and is ≤ 0.

Key takeaway. Equilibrium: the discriminator outputs $\frac{1}{2}$ everywhere, $V = -\ln 4$.

Q.73 SINGLE-CORRECT MCQ · Q-learning update

In (quantum) reinforcement learning with Q-learning, $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$. With $Q(s,a) = 2$, $\alpha = 0.5$, reward $r = 1$, discount $\gamma = 0.9$ and $\max_{a'} Q(s',a') = 4$, the updated value is

(A) 3.5 **(B)** 2.6 **(C)** ✓ 3.3 **(D)** 4.6

Answer: (C) Detailed solution

Correct answer. (C) 3.3

Concepts required. Temporal-difference (Q-learning) update.

Relevant formulae. $Q \leftarrow Q + \alpha(r + \gamma \max_{a'} Q' - Q)$.

Step-by-step explanation. Target $= r + \gamma \max Q' = 1 + 0.9 \times 4 = 4.6$; temporal-difference error $= 4.6 - 2 = 2.6$; update $= 2 + 0.5 \times 2.6 = 3.3$. In quantum RL a variational circuit can represent the Q-function or the policy $\pi(a \mid s) = |\langle a|U(s;\boldsymbol{\theta})|0\rangle|^2$; the update rule itself is classical.

Why the other options are incorrect.

(A) 3.5 ignores the discount ($\gamma = 1$).

(B) 2.6 is the TD error, not the new Q.

(D) 4.6 is the target $r + \gamma \max Q'$, not the updated value (it would need $\alpha = 1$).

Key takeaway. New Q = old Q + learning rate × TD error.

Q.74 Single-correct MCQ · Swap-test distances (clustering)

A quantum clustering algorithm estimates distances between normalised real data vectors $\mathbf{a}, \mathbf{b}$ with a swap test. The ancilla gives 0 with probability 0.9, and $\mathbf{a} \cdot \mathbf{b} > 0$ is known. The Euclidean distance $\|\mathbf{a} - \mathbf{b}\|$ is about

(A) 0.21 **(B)** ✓ 0.46 **(C)** 0.63 **(D)** 0.89

Answer: (B) Detailed solution

Correct answer. (B) 0.46
Concepts required. Distances from overlaps; swap test estimates $|\langle a|b\rangle|^2$.
Relevant formulae. $|\mathbf{a} \cdot \mathbf{b}|^2 = 2P(0) - 1$; $\|\mathbf{a} - \mathbf{b}\|^2 = 2 - 2\mathbf{a} \cdot \mathbf{b}$ for unit vectors.
Step-by-step explanation. Swap test: $|\mathbf{a} \cdot \mathbf{b}|^2 = 2P(0) - 1 = 0.8$, so $\mathbf{a} \cdot \mathbf{b} = \sqrt{0.8} = 0.894$. For unit vectors $\|\mathbf{a} - \mathbf{b}\|^2 = 2 - 2\,\mathbf{a} \cdot \mathbf{b} = 2 - 1.789 = 0.211$, hence $\|\mathbf{a} - \mathbf{b}\| = 0.46$. (Using 0.8 as the overlap gives $\sqrt{0.4} = 0.63$.) The swap test only gives $|\mathbf{a} \cdot \mathbf{b}|$; the sign must be known or obtained otherwise (e.g. a Hadamard test).
Why the other options are incorrect.
(A) 0.21 is the squared distance.
(C) 0.63 uses 0.8 (the squared overlap) as the overlap.
(D) 0.89 is the overlap $\mathbf{a} \cdot \mathbf{b}$, not the distance.
Key takeaway. Overlap → distance: take the square root of $2P(0) - 1$ first.

Q.75 Matching lists · CUDA-Q targets

Match each CUDA-Q target (List-I) with its use (List-II).

List-I		List-II	
(P)	`qpp-cpu`	**(1)**	one state vector distributed over several GPUs (one MPI rank per GPU) to simulate more qubits
(Q)	`nvidia` with option `mgpu`	**(2)**	several independent virtual QPUs (one per GPU) that run *different* circuits in parallel, e.g. many images or shift terms
(R)	`nvidia` with option `mqpu`	**(3)**	CPU state-vector simulator; produced the recorded outputs of `QML.ipynb`
(S)	`density-matrix-cpu`	**(4)**	noisy simulation with a noise model; memory $16 \cdot 4^N$ bytes

(A) P→3, Q→1, R→4, S→2 **(B)** P→4, Q→1, R→2, S→3
(C) ✓ P→3, Q→1, R→2, S→4 **(D)** P→3, Q→2, R→1, S→4

Answer: (C) Detailed solution

Correct answer. (C) P→3, Q→1, R→2, S→4
Concepts required. CUDA-Q simulation targets and their use in QML workloads.
Relevant formulae. State vector: $16 \cdot 2^N$ B (`qpp-cpu`, `nvidia`); distributed: $16 \cdot 2^N/G$ per GPU (`mgpu`); density matrix: $16 \cdot 4^N$ B.
Step-by-step explanation.

- `qpp-cpu`: the default when no GPU is visible; the notebook's recorded run used it (`GPUs`

`visible to CUDA-Q: 0`) (3).

- `mgpu` pools the memory of several GPUs for *one* large state vector; it needs `mpiexec` (1).
- `mqpu` exposes each GPU as a separate QPU for asynchronous, parallel execution of many *small* circuits — the natural choice for QML, where each training step evaluates one circuit per image (2).
- `density-matrix-cpu` stores ρ (4^N entries) and applies noise channels (4).

Hence P→3, Q→1, R→2, S→4. `QML.ipynb` itself selects only `nvidia` or `qpp-cpu` (or a user-given name) and uses neither `mgpu` nor `mqpu`.

Why the other options are incorrect.

(A) R→4, S→2: `mqpu` is a parallel noiseless state-vector mode; noise needs a density-matrix or trajectory target.

(B) P→4, S→3 swaps the CPU state-vector and density-matrix simulators.

(D) Q→2, R→1 swaps `mgpu` (one big state) and `mqpu` (many small circuits in parallel).

Key takeaway. QML batches many small circuits: `mqpu` parallelises them; `mgpu` only helps for very wide circuits.

Q.76 MATCHING LISTS · Limitations of QML

Match each limitation of QML (List-I) with its description (List-II).

List-I		List-II	
(P)	barren plateau	**(1)**	classical sampling algorithms reproduce claimed exponential speed-ups (e.g. recommendation systems, PCA) under the same input assumptions
(Q)	dequantization	**(2)**	the variance of cost gradients decays exponentially with the number of qubits
(R)	data-loading bottleneck	**(3)**	off-diagonal kernel values approach a constant exponentially fast, so exponentially many shots are needed to distinguish them
(S)	exponential kernel concentration	**(4)**	preparing a general amplitude-encoded state of 2^n numbers needs $O(2^n)$ gates, which can cancel a speed-up

(A) P→2, Q→4, R→1, S→3 **(B)** P→1, Q→2, R→4, S→3

(C) P→3, Q→1, R→4, S→2 **(D)** ✓ P→2, Q→1, R→4, S→3

Answer: (D) Detailed solution

Correct answer. (D) P→2, Q→1, R→4, S→3

Concepts required. The main known obstacles to useful QML.

Relevant formulae. BP: $\mathrm{Var}[\partial C] \sim 2^{-n}$; concentration: $|k - \bar{k}| \sim c^n$; loading: $O(2^n)$ gates for general amplitude states.

Step-by-step explanation.

- Barren plateau (McClean et al. 2018): $\mathrm{Var}[\partial C] \sim 2^{-n}$ (2).
- Dequantization (Tang 2019 and later): quantum-inspired classical algorithms with comparable sampling access (1).
- Data loading: amplitude encoding is qubit-cheap but gate-expensive; QRAM is not available

at scale (4).

- Kernel concentration (Thanasilp et al. 2022): $k(\mathbf{x}, \mathbf{x}') \to$ const, the model cannot generalise with polynomial shots (3).

Hence P→2, Q→1, R→4, S→3.

Why the other options are incorrect.

(A) Q→4, R→1 swaps dequantization and data loading.

(B) P→1, Q→2 swaps barren plateaus and dequantization.

(C) P→3, S→2 swaps gradient and kernel concentration; both are exponential, but in different objects.

Key takeaway. Trainability, classical competition and data loading must all be addressed before claiming advantage.

Q.77 Assertion–Reason · Quantum-advantage claims

Assertion (A): Because the QCNN of `QML.ipynb` reaches 85 % test accuracy, it demonstrates a quantum advantage over classical image classifiers.

Reason (R): Before any advantage can be claimed, the quantum model must be compared with classical baselines trained and evaluated on the same data, at comparable cost.

(A) Both Assertion (A) and Reason (R) are true, and R is the correct explanation of A

(B) Both A and R are true, but R is *not* the correct explanation of A

(C) A is true, but R is false

(D) ✓ A is false, but R is true

Answer: (D) Detailed solution

Correct answer. **(D)** A is false, but R is true

Concepts required. Standards of evidence for quantum advantage in ML.

Relevant formulae. Compare with strong classical baselines on the same data, with uncertainty (SE $\approx \sqrt{a(1-a)/M}$) and cost.

Step-by-step explanation. **A is false:** 85 % on 20 test images (±8 points) only shows that the model learned something; the notebook itself states that a classical CNN or even simple methods solve the toy task easily, and the circuit was *simulated* classically in 3.4 s. In the lecture's experiments a hand-written rule reached 100 % and a classically computable kernel SVM 95.2 % on fresh images (QCNN 80.2 %). **R is true**: it is the minimum standard of benchmarking. Answer: (D).

Why the other options are incorrect.

(A) needs A true; one accuracy number on a toy task is not evidence of advantage.

(B) needs A true as well.

(C) needs R false, but baseline comparison is the minimum requirement.

Key takeaway. No baseline, no claim: and here simple classical methods do better.

Q.78 STATEMENT I / II · Recorded hardware; target selection

Statement I: The outputs stored in `QML.ipynb` were produced on an NVIDIA GPU.

Statement II: With `TARGET = "auto"` on a machine with two GPUs, `QML.ipynb` distributes the QCNN state vector over both GPUs.

(A) Both Statement I and Statement II are true

(B) ✓ Both Statement I and Statement II are false

(C) Statement I is true, but Statement II is false

(D) Statement I is false, but Statement II is true

Answer: (B) Detailed solution

Correct answer. (B) Both Statement I and Statement II are false

Concepts required. Reading the recorded outputs and the target-selection code of `QML.ipynb`.

Relevant formulae. `TARGET = "auto"` → `"nvidia"` if `cudaq.num_available_gpus() > 0` else `"qpp-cpu"`; no options passed.

Step-by-step explanation. **I is false:** the recorded output prints `GPUs visible to CUDA-Q: 0` and `CUDA-Q target: qpp-cpu`; the README states the run was on a CPU-only container, and GPU paths were not executed. **II is false:** `"auto"` selects `"nvidia"` when a GPU is visible, via `cudaq.set_target(target_name)` without options — a single-GPU state vector. Distributing one state would need the `mgpu` option and MPI; running images in parallel would need `mqpu`. Answer: (B).

Why the other options are incorrect.

(A) needs both true; neither is.

(C) needs I true, but the recorded run shows 0 GPUs and `qpp-cpu`.

(D) needs II true, but `set_target("nvidia")` without `mgpu` uses one GPU.

Key takeaway. Report hardware exactly as recorded; GPU and multi-GPU paths of the QML notebook were not executed.

Paragraph for Questions 79 and 80

For the lecture, `QML.ipynb` was re-executed with its default settings (CUDA-Q 0.16.0, `qpp-cpu`, no GPU). It reproduced the recorded outputs exactly: loss 1.1214 → 0.6704, training accuracy 90 %, test accuracy 85 %. With "horizontal" as the positive class, the 20 test images give the confusion matrix

	predicted horizontal	predicted vertical
true horizontal (10 images)	9	1
true vertical (10 images)	2	8

Further measurements on the *same* 20 training images (illustrative experiments, not part of the notebook): logistic regression on the 8 raw pixels reaches 45 % test accuracy; a hand-written rule ("is some horizontal pixel pair brighter than every vertical pixel pair?") reaches 100 %; an SVM with the fidelity kernel of the notebook's angle encoding reaches 100 %. On 2000 fresh images from the same generator the accuracies are: trained QCNN 80.2 %, kernel SVM 95.2 %, hand-written rule 100 %, logistic regression 46.5 %.

Q.79 PARAGRAPH BASED · Recorded run: precision, recall, F1

For the positive class "horizontal", the precision, recall and F1-score of the trained QCNN on the 20 test images are

(A) 0.900, 0.818, 0.857
(B) 0.850, 0.850, 0.850
(C) ✓ 0.818, 0.900, 0.857
(D) 0.818, 0.900, 0.859

Answer: (C) Detailed solution

Correct answer. **(C)** 0.818, 0.900, 0.857
Concepts required. Confusion-matrix metrics for a chosen positive class.
Relevant formulae. $P = \frac{TP}{TP+FP}$, $R = \frac{TP}{TP+FN}$, $F_1 = \frac{2TP}{2TP+FP+FN}$.
Step-by-step explanation. From the table: $TP = 9$, $FN = 1$, $FP = 2$, $TN = 8$. Precision $= \frac{TP}{TP+FP} = \frac{9}{11} = 0.818$; recall $= \frac{TP}{TP+FN} = \frac{9}{10} = 0.900$; $F_1 = \frac{2TP}{2TP+FP+FN} = \frac{18}{21} = 0.857$ (the harmonic mean, slightly below the arithmetic mean 0.859). Accuracy $= \frac{9+8}{20} = 0.85$ agrees with the recorded test accuracy.
Why the other options are incorrect.
(A) Precision and recall are swapped (9/10 is recall, 9/11 precision).
(B) 0.85 is the accuracy, reported for all three metrics.
(D) 0.859 is the arithmetic mean of P and R, not F1.
Key takeaway. TP 9, FP 2, FN 1: $P = 0.818$, $R = 0.900$, $F_1 = 0.857$.

Q.80 PARAGRAPH BASED · Recorded run vs. classical baselines

Which conclusion is supported by these measurements?

(A) The drop from 85 % to 80.2 % proves that the 2000 fresh images come from a different distribution than the training images.
(B) The kernel SVM result requires a quantum computer, because its kernel is defined as an overlap of quantum states.
(C) The QCNN demonstrates a quantum advantage, because it beats logistic regression by about 35 points.
(D) ✓ On fresh data the QCNN (80.2 %) is below the classically computable kernel SVM (95.2 %) and the simple rule (100 %); its 85 % on 20 images lies within about one standard error (≈ 8 points) of 80 %; logistic regression fails because the classes are not linearly separable in pixel space.

Answer: (D) Detailed solution

Correct answer. **(D)** On fresh data the QCNN (80.2 %) is below the classically computable kernel SVM (95.2 %) and the simple rule (100 %); its 85 % on 20 images lies within about one standard error (≈ 8 points) of 80 %; logistic regression fails because the classes are not linearly separable in pixel space.
Concepts required. Benchmarking: larger test sets, strong baselines, classical simulability of the quantum kernel, linear separability.
Relevant formulae. SE $= \sqrt{a(1-a)/M}$; angle-encoding kernel $= \prod_i \cos^2 \frac{\pi(x_i - x_i')}{2}$; for clean images the constraints of the horizontal pairs in columns 1–2 and the vertical lines in columns 1 and 2 sum to $s + 2b > 0$ and $s + 2b < 0$: not linearly separable.
Step-by-step explanation. The fresh test set (2000 images) has a standard error of only $\sqrt{0.8 \cdot 0.2/2000} \approx 0.9$ points, so it measures the QCNN's generalisation much better than 20 images (≈ 8 points); 85 % vs. 80.2 % is consistent with sampling noise, and both sets come from the same generator. The angle-encoding kernel is $\prod_i \cos^2 \frac{\pi(x_i - x_i')}{2}$, computable classically in $O(d)$; the rule needs no training at all. Beating one weak (linear) baseline is not evidence of advantage: a line can

be horizontal or vertical at any position, and no single linear function of the pixels separates the two classes well.

Why the other options are incorrect.

(A) Both test sets come from the same generator; 85 % vs. 80.2 % is within the ±8-point error of 20 images.

(B) The kernel has a closed classical formula; it was in fact computed with both CUDA-Q states and the formula (difference 10^{-15}).

(C) Beating one weak linear baseline is not advantage; stronger classical methods (rule, kernel SVM) beat the QCNN.

Key takeaway. Bigger test sets and strong baselines turn a promising number into an honest assessment.

SILICO FELLER